KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

Academic Year 2026–2027

WEB TECHNOLOGY

ASSIGNMENT

Course Code: U21CS501 | Maximum Marks: 40

Student Name	Amar Kumar Pandit
Roll Number	24CS255
Course Code	U21CS501
Course Title	Web Technology
Department	Computer Science Engineering (CSE)
Year / Semester	III Year / Semester V
Academic Year	2026–2027
Submission Date	14 October 2026

Q1: Registration Form (HTML/CSS/JS) • Q2: PHP + MySQL Student Management System
Q3: Java Servlet / JSP / JDBC Student Profile Portal

Student Signature

Staff Signature

Marks Obtained: _____ / 40

Table of Contents

Question 1 — Student Registration Form (8 Marks) 4

1. Aim 4
2. Software Requirements 4
3. Algorithm / Procedure 4
4. HTML Source Code (index.html) 5
5. CSS Source Code (style.css) 6
6. JavaScript Source Code (script.js) 8
7. Explanation of the Code 11
8. Validation of Fields 11
9. Result 12
10. Output Screenshots 12

Question 2 — PHP Student Management System (16 Marks) 13

1. Aim 13
2. Software Requirements 13
3. Database Design 13
4. SQL Database Creation Script 14
5. Algorithm / Procedure 15
6. Project Structure 15
7. PHP Database Connection 16
8. Helper Functions: Validation, CSRF and Secure File Upload 16
9. Common Layout and Shared Form 20
10. Add Student 21
11. View Students (Dashboard and List) 22
12. Edit Student 24
13. Delete Student 26
14. Student Details Page 27
15. Display and Download of Uploaded Files 28
16. Stylesheet and JavaScript 29
17. File Validation and Error Handling 31
18. Explanation of Important Code 31
19. Result 32
20. Output Screenshots 32

Question 3 — Java Servlet Student Profile Portal (16 Marks) 35

1. Aim . . . 35
2. Software Requirements . . . 35
3. System Architecture . . . 35
4. Database Design and SQL Script . . . 36
5. Algorithm / Procedure . . . 37
6. pom.xml and Required Configuration . . . 37
7. JDBC Database Connection Class . . . 40
8. Password Hashing Utility . . . 41
9. Student Model and DAO Classes . . . 42
10. Login Servlet and Session Handling . . . 44
11. Authentication Filter (Session Check) . . . 46
12. Student List Servlet . . . 47
13. JSP Pages: Dynamic Profiles, Looping and Conditional Logic . . . 47
14. Image and Document Streaming Servlet . . . 50
15. Logout Servlet . . . 52
16. Stylesheet . . . 52
17. Explanation of the Code . . . 53
18. Result . . . 54
19. Output Screenshots . . . 54

Conclusion . . . 57

Question 1 — Student Registration Form (8 Marks)

1. Aim

To design and develop a responsive **Student Registration Form** using HTML, CSS and JavaScript with client-side validation of required fields, e-mail format, password strength, password confirmation and file uploads.

2. Software Requirements

Component	Requirement
Operating system	Windows / Linux / macOS
Web browser	Google Chrome, Microsoft Edge or Mozilla Firefox (latest version)
Code editor	Visual Studio Code (or any text editor)
Languages	HTML5, CSS3, JavaScript (ES6)
Server	Not required – the page runs directly in the browser

3. Algorithm / Procedure

1. Start.
2. Create `index.html` containing a form with fields for student name, roll number, e-mail, course (drop-down), password, confirm password and file upload, plus Register and Reset buttons.
3. Add an empty `<small>` element under every field to display its error message, and a strength bar under the password field.
4. Create `style.css` to style the card layout, inputs, buttons, valid / invalid states, the strength bar and the success message; add a media query for small screens.
5. Create `script.js`; read all form elements using `document.getElementById`.
6. Write one validation function per field. Each function trims the value, checks it against its rule (empty, regular expression, size, type) and either shows an error or marks the field valid.
7. Attach `input` / `change` listeners so each field is validated while the user types, and update the password strength meter on every key press.
8. On form `submit`, call `preventDefault()`, run all validators and collect the results.
9. If every validator returns true, display the success message; otherwise keep the error messages visible and move focus to the first invalid field.
10. On `reset`, clear all messages and styles. Stop.

4. HTML Source Code (index.html)

question1/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration Form</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Student Registration Form</h1>
    <p class="subtitle">KPR Institute of Engineering and Technology</p>

    <!-- novalidate: browser's default popups are disabled; script.js validates -->
    <form id="registrationForm" novalidate>

      <!-- Student name -->
      <div class="form-group">
        <label for="studentName">Student Name <span class="req">*</span></label>
        <input type="text" id="studentName" name="studentName" placeholder="Enter full name">
        <small class="error" id="studentNameError"></small>
      </div>

      <!-- Roll number -->
      <div class="form-group">
        <label for="rollNumber">Roll Number <span class="req">*</span></label>
        <input type="text" id="rollNumber" name="rollNumber" placeholder="e.g. 24CS255">
        <small class="error" id="rollNumberError"></small>
      </div>

      <!-- Email -->
      <div class="form-group">
        <label for="email">Email <span class="req">*</span></label>
        <input type="email" id="email" name="email" placeholder="name@example.com">
        <small class="error" id="emailError"></small>
      </div>

      <!-- Course -->
      <div class="form-group">
        <label for="course">Course <span class="req">*</span></label>
        <select id="course" name="course">
          <option value="">-- Select Course --</option>
          <option value="CSE">B.E. Computer Science and Engineering</option>
          <option value="IT">B.Tech Information Technology</option>
          <option value="ECE">B.E. Electronics and Communication</option>
          <option value="EEE">B.E. Electrical and Electronics</option>
          <option value="MECH">B.E. Mechanical Engineering</option>
        </select>
        <small class="error" id="courseError"></small>
      </div>

      <!-- Password with strength meter -->
      <div class="form-group">
        <label for="password">Password <span class="req">*</span></label>
        <input type="password" id="password" name="password" placeholder="Min 8 chars, upper, lower, digit,
            symbol">
        <div class="strength-bar"><div id="strengthFill"></div></div>
        <small id="strengthText" class="strength-text"></small>
        <small class="error" id="passwordError"></small>
      </div>

      <!-- Confirm password -->
      <div class="form-group">
        <label for="confirmPassword">Confirm Password <span class="req">*</span></label>
        <input type="password" id="confirmPassword" name="confirmPassword" placeholder="Re-enter password">
        <small class="error" id="confirmPasswordError"></small>
      </div>

      <!-- File upload -->
      <div class="form-group">
        <label for="fileUpload">Upload Photo / Document <span class="req">*</span></label>
        <input type="file" id="fileUpload" name="fileUpload" accept=".jpg,.jpeg,.png,.pdf">
        <small class="hint">Allowed: JPG, PNG, PDF. Maximum size 2 MB.</small>
        <small class="error" id="fileUploadError"></small>
      </div>
```

question1/index.html (continued)

```
      <div class="button-row">
        <button type="submit" class="btn btn-primary">Register</button>
        <button type="reset" class="btn btn-secondary" id="resetBtn">Reset</button>
      </div>

      <div id="successMessage" class="success" hidden></div>
    </form>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

5. CSS Source Code (style.css)

question1/style.css

```
/* ---------- Global reset and page layout ---------- */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: "Segoe UI", Arial, sans-serif;
  background: linear-gradient(135deg, #e8eef9, #f6f8fc);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  padding: 30px 15px;
  color: #1f2937;
}

/* ---------- Card container ---------- */
.container {
  width: 100%;
  max-width: 520px;
  background: #ffffff;
  border-radius: 10px;
  padding: 30px 32px;
  box-shadow: 0 6px 24px rgba(15, 23, 42, 0.12);
  border-top: 5px solid #1e3a8a;
}

h1 {
  text-align: center;
  color: #1e3a8a;
  font-size: 1.7rem;
}

.subtitle {
  text-align: center;
  color: #6b7280;
  margin: 4px 0 22px;
  font-size: 0.9rem;
}

/* ---------- Form controls ---------- */
.form-group {
  margin-bottom: 16px;
}

label {
  display: block;
  font-weight: 600;
  margin-bottom: 5px;
  font-size: 0.92rem;
}

.req {
  color: #dc2626;
}
```

question1/style.css (continued)

```
input[type="text"],
input[type="email"],
input[type="password"],
input[type="file"],
select {
  width: 100%;
  padding: 10px 12px;
  border: 1.5px solid #cbd5e1;
  border-radius: 6px;
  font-size: 0.95rem;
  background: #f9fafb;
  transition: border-color 0.2s, box-shadow 0.2s;
}

input:focus,
select:focus {
  outline: none;
  border-color: #2563eb;
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.18);
}

/* ---------- Validation states (classes toggled by script.js) ---------- */
input.invalid,
select.invalid {
  border-color: #dc2626;
  background: #fef2f2;
}

input.valid,
select.valid {
  border-color: #16a34a;
}

.error {
  display: block;
  color: #dc2626;
  font-size: 0.8rem;
  min-height: 1em;
  margin-top: 4px;
}

.hint {
  display: block;
  color: #6b7280;
  font-size: 0.78rem;
  margin-top: 4px;
}

/* ---------- Password strength meter ---------- */
.strength-bar {
  height: 6px;
  background: #e5e7eb;
  border-radius: 4px;
  margin-top: 8px;
  overflow: hidden;
}

#strengthFill {
  height: 100%;
  width: 0;
  transition: width 0.3s, background 0.3s;
}

.strength-text {
  display: block;
  font-size: 0.8rem;
  margin-top: 4px;
  font-weight: 600;
}

/* ---------- Buttons ---------- */
.button-row {
  display: flex;
  gap: 12px;
  margin-top: 22px;
}
```

question1/style.css (continued)

```
.btn {
  flex: 1;
  padding: 11px;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s;
}

.btn-primary {
  background: #1e3a8a;
  color: #ffffff;
}

.btn-primary:hover {
  background: #1d4ed8;
}

.btn-secondary {
  background: #e5e7eb;
  color: #374151;
}

.btn-secondary:hover {
  background: #d1d5db;
}

/* ---------- Success message ---------- */
.success {
  margin-top: 18px;
  padding: 12px 14px;
  background: #dcfce7;
  border: 1px solid #16a34a;
  color: #166534;
  border-radius: 6px;
  font-weight: 600;
  text-align: center;
}

/* ---------- Small screens ---------- */
@media (max-width: 480px) {
  .container {
    padding: 22px 18px;
  }
}
```

6. JavaScript Source Code (script.js)

question1/script.js

```
// script.js - client-side validation for the Student Registration Form
"use strict";

// ---------- Cached DOM references ----------
const form = document.getElementById("registrationForm");
const fields = {
  studentName: document.getElementById("studentName"),
  rollNumber: document.getElementById("rollNumber"),
  email: document.getElementById("email"),
  course: document.getElementById("course"),
  password: document.getElementById("password"),
  confirmPassword: document.getElementById("confirmPassword"),
  fileUpload: document.getElementById("fileUpload")
};
const successBox = document.getElementById("successMessage");
const strengthFill = document.getElementById("strengthFill");
const strengthText = document.getElementById("strengthText");

// ---------- Validation rules ----------
const NAME_REGEX = /^[A-Za-z][A-Za-z .'-]{2,49}$/;            // letters/spaces, 3-50 chars
const ROLL_REGEX = /^[0-9]{2}[A-Za-z]{2,4}[0-9]{1,4}$/;       // e.g. 24CS255
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;          // simple email pattern
const MAX_FILE_SIZE = 2 * 1024 * 1024;                         // 2 MB
const ALLOWED_TYPES = ["image/jpeg", "image/png", "application/pdf"];
const ALLOWED_EXTENSIONS = ["jpg", "jpeg", "png", "pdf"];
```

question1/script.js (continued)

```
// ---------- Helper functions to show / clear messages ----------
function showError(fieldName, message) {
  const input = fields[fieldName];
  document.getElementById(fieldName + "Error").textContent = message;
  input.classList.add("invalid");
  input.classList.remove("valid");
  return false;
}

function showValid(fieldName) {
  const input = fields[fieldName];
  document.getElementById(fieldName + "Error").textContent = "";
  input.classList.remove("invalid");
  input.classList.add("valid");
  return true;
}

// ---------- Individual field validators (each returns true/false) ----------
function validateName() {
  const value = fields.studentName.value.trim();
  if (value === "") return showError("studentName", "Student name is required.");
  if (!NAME_REGEX.test(value)) {
    return showError("studentName", "Use 3-50 letters (spaces, . ' - allowed).");
  }
  return showValid("studentName");
}

function validateRoll() {
  const value = fields.rollNumber.value.trim();
  if (value === "") return showError("rollNumber", "Roll number is required.");
  if (!ROLL_REGEX.test(value)) {
    return showError("rollNumber", "Invalid roll number format (example: 24CS255).");
  }
  return showValid("rollNumber");
}

function validateEmail() {
  const value = fields.email.value.trim();
  if (value === "") return showError("email", "Email is required.");
  if (!EMAIL_REGEX.test(value)) {
    return showError("email", "Enter a valid email address (name@domain.com).");
  }
  return showValid("email");
}

function validateCourse() {
  if (fields.course.value === "") {
    return showError("course", "Please select a course.");
  }
  return showValid("course");
}

// Returns a score from 0 to 5 based on the rules satisfied
function passwordScore(pwd) {
  let score = 0;
  if (pwd.length >= 8) score++;
  if (/[a-z]/.test(pwd)) score++;
  if (/[A-Z]/.test(pwd)) score++;
  if (/[0-9]/.test(pwd)) score++;
  if (/[^A-Za-z0-9]/.test(pwd)) score++;
  return score;
}

function validatePassword() {
  const pwd = fields.password.value;
  if (pwd === "") return showError("password", "Password is required.");
  if (pwd.length < 8) return showError("password", "Password must be at least 8 characters.");
  if (!/[a-z]/.test(pwd)) return showError("password", "Add at least one lowercase letter.");
  if (!/[A-Z]/.test(pwd)) return showError("password", "Add at least one uppercase letter.");
  if (!/[0-9]/.test(pwd)) return showError("password", "Add at least one digit.");
  if (!/[^A-Za-z0-9]/.test(pwd)) return showError("password", "Add at least one special character.");
  return showValid("password");
}
```

question1/script.js (continued)

```
function validateConfirmPassword() {
  const confirm = fields.confirmPassword.value;
  if (confirm === "") return showError("confirmPassword", "Please confirm your password.");
  if (confirm !== fields.password.value) {
    return showError("confirmPassword", "Passwords do not match.");
  }
  return showValid("confirmPassword");
}

function validateFile() {
  const file = fields.fileUpload.files[0];
  if (!file) return showError("fileUpload", "Please choose a file to upload.");

  const extension = file.name.split(".").pop().toLowerCase();
  if (!ALLOWED_EXTENSIONS.includes(extension) || !ALLOWED_TYPES.includes(file.type)) {
    return showError("fileUpload", "Only JPG, PNG or PDF files are allowed.");
  }
  if (file.size > MAX_FILE_SIZE) {
    return showError("fileUpload", "File is too large (maximum 2 MB).");
  }
  return showValid("fileUpload");
}

// ---------- Password strength meter ----------
function updateStrengthMeter() {
  const pwd = fields.password.value;
  const score = pwd === "" ? 0 : passwordScore(pwd);
  const levels = [
    { text: "",            color: "#e5e7eb", width: "0%" },
    { text: "Very Weak",   color: "#dc2626", width: "20%" },
    { text: "Weak",        color: "#f97316", width: "40%" },
    { text: "Fair",        color: "#eab308", width: "60%" },
    { text: "Strong",      color: "#65a30d", width: "80%" },
    { text: "Very Strong", color: "#16a34a", width: "100%" }
  ];
  const level = levels[score];
  strengthFill.style.width = level.width;
  strengthFill.style.background = level.color;
  strengthText.textContent = level.text ? "Strength: " + level.text : "";
  strengthText.style.color = level.color;
}

// ---------- Live validation while the user types ----------
fields.studentName.addEventListener("input", validateName);
fields.rollNumber.addEventListener("input", validateRoll);
fields.email.addEventListener("input", validateEmail);
fields.course.addEventListener("change", validateCourse);
fields.password.addEventListener("input", function () {
  updateStrengthMeter();
  validatePassword();
  if (fields.confirmPassword.value !== "") validateConfirmPassword();
});
fields.confirmPassword.addEventListener("input", validateConfirmPassword);
fields.fileUpload.addEventListener("change", validateFile);

// ---------- Form submission ----------
form.addEventListener("submit", function (event) {
  event.preventDefault();           // stop the page from reloading
  successBox.hidden = true;

  // Run every validator (no short-circuit, so all errors are shown at once)
  const results = [
    validateName(),
    validateRoll(),
    validateEmail(),
    validateCourse(),
    validatePassword(),
    validateConfirmPassword(),
    validateFile()
  ];
```

question1/script.js (continued)

```
  if (results.every(Boolean)) {
    successBox.textContent =
      "Registration successful for " + fields.studentName.value.trim() + "!";
    successBox.hidden = false;
    // In a real application the data would now be sent to a server.
  } else {
    // Move focus to the first invalid field
    const firstInvalid = form.querySelector(".invalid");
    if (firstInvalid) firstInvalid.focus();
  }
});

// ---------- Reset handler: clear messages and styles ----------
form.addEventListener("reset", function () {
  Object.keys(fields).forEach(function (name) {
    document.getElementById(name + "Error").textContent = "";
    fields[name].classList.remove("invalid", "valid");
  });
  successBox.hidden = true;
  // Wait until the browser clears the values, then refresh the meter
  setTimeout(updateStrengthMeter, 0);
});
```

7. Explanation of the Code

• **index.html** – defines the form structure. `novalidate` switches off the browser's default pop-ups so that our own messages are shown. Every input has a `<label>` linked by `for`/`id` for accessibility, and a `<small class="error">` placeholder for its message.

• **style.css** – a centred white card on a soft background. The classes `.invalid` (red border) and `.valid` (green border) are added or removed by JavaScript. The strength bar width and colour are changed from script. The media query reduces padding on mobile screens.

• **script.js** – `showError()` and `showValid()` are shared helpers that write the message and toggle the CSS classes. Each `validateXxx()` function returns `true` or `false`. Regular expressions are kept in constants at the top so they are easy to change.

• **Live validation** – `input` and `change` event listeners call the validators as the user types; the password listener also refreshes the strength meter and re-checks the confirm-password field.

• **Submit handling** – all validators are executed (the results array avoids short-circuiting) so every error is shown together; `results.every(Boolean)` decides success.

• **Reset handling** – clears messages, red/green borders and the success box; `setTimeout` waits for the browser to clear the values before the meter is refreshed.

8. Validation of Fields

Field	Validation rule	Message shown on failure
Student name	Required; 3–50 letters, spaces and . ' - allowed	Student name is required. / Use 3-50 letters...
Roll number	Required; pattern `^[0-9]{2}[A-Za-z]{2,4}[0-9]{1,4}$` (e.g. 24CS255)	Invalid roll number format (example: 24CS255).
E-mail	Required; pattern `name@domain.tld`	Enter a valid email address (name@domain.com).
Course	A course must be selected	Please select a course.
Password	Min. 8 characters with lowercase, uppercase, digit and special character; live strength meter (Very Weak to Very Strong)	Specific message for the first rule that is not met.

Field	Validation rule	Message shown on failure
Confirm password	Required and must equal the password	Passwords do not match.
File upload	Required; extension and MIME type must be JPG, PNG or PDF; size ≤ 2 MB	Only JPG, PNG or PDF files are allowed. / File is too large (maximum 2 MB).

9. Result

The Student Registration Form was designed and implemented using HTML, CSS and JavaScript. The program is written to validate the name, roll number, e-mail, course, password strength, password confirmation and uploaded file on the client side, to show an error message beside every invalid field, and to display a success message only when all the checks pass.

10. Output Screenshots

Open `question1/index.html` in a browser, then paste your own screenshots in the boxes below.

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 1.1 – Registration form with validation errors

Figure 1.1 – Registration form with validation errors

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 1.2 – Registration form after successful submission

Figure 1.2 – Registration form after successful submission

Question 2 — PHP Student Management System (16 Marks)

1. Aim

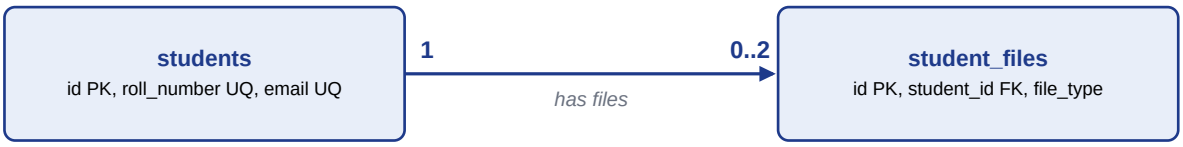
To develop a **Student Management System** using PHP and MySQL that performs Create, Read, Update and Delete (CRUD) operations on student records and supports secure upload, display and download of a profile image and a PDF document for every student.

2. Software Requirements

Component	Requirement
Web stack	XAMPP / WAMP / LAMP (Apache 2.4 + PHP 7.4 or later + MySQL 5.7 / MariaDB 10.x)
PHP extensions	`mysqli`, `fileinfo` (both enabled by default in XAMPP)
Front end	HTML5, CSS3, JavaScript (ES6)
Database tool	phpMyAdmin or MySQL command-line client
Editor / Browser	Visual Studio Code; Chrome, Edge or Firefox

3. Database Design

The database `student_management` has two tables in a one-to-many relationship. File contents are kept on disk; only their metadata (original name, unique stored name, MIME type, size) is stored in MySQL.



ON DELETE CASCADE: deleting a student removes its file records

Table: students

Column	Type	Constraints / purpose
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT
student_name	VARCHAR(100)	NOT NULL
roll_number	VARCHAR(20)	NOT NULL, UNIQUE
email	VARCHAR(120)	NOT NULL, UNIQUE
phone	VARCHAR(15)	NULL allowed
course	VARCHAR(60)	NOT NULL
created_at / updated_at	TIMESTAMP	set automatically

Table: student_files (file metadata)

Column	Type	Constraints / purpose
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT
student_id	INT UNSIGNED	FOREIGN KEY → students.id, ON DELETE CASCADE
file_type	ENUM('image','document')	UNIQUE together with student_id (one image + one document per student)
original_name	VARCHAR(255)	name shown to the user
stored_name	VARCHAR(100)	UNIQUE random file name on disk
mime_type, file_size	VARCHAR(100), INT	verified MIME type and size in bytes
uploaded_at	TIMESTAMP	set automatically

4. SQL Database Creation Script

question2/sql/database.sql

```
-- database.sql : Student Management System (Question 2)
-- Run with:  mysql -u root -p < sql/database.sql

CREATE DATABASE IF NOT EXISTS student_management
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE student_management;

-- ------------------------------------------------------
-- Table 1: students (core student data)
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS students (
  id            INT UNSIGNED NOT NULL AUTO_INCREMENT,
  student_name  VARCHAR(100) NOT NULL,
  roll_number   VARCHAR(20)  NOT NULL,
  email         VARCHAR(120) NOT NULL,
  phone         VARCHAR(15)  DEFAULT NULL,
  course        VARCHAR(60)  NOT NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_students_roll (roll_number),
  UNIQUE KEY uq_students_email (email)
) ENGINE=InnoDB;

-- ------------------------------------------------------
-- Table 2: student_files (metadata of uploaded files)
-- One student can have one profile image and one document.
-- The file itself lives on disk; only metadata is stored here.
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS student_files (
  id             INT UNSIGNED NOT NULL AUTO_INCREMENT,
  student_id     INT UNSIGNED NOT NULL,
  file_type      ENUM('image','document') NOT NULL,
  original_name  VARCHAR(255) NOT NULL,   -- name shown to the user
  stored_name    VARCHAR(100) NOT NULL,   -- unique random name on disk
  mime_type      VARCHAR(100) NOT NULL,
  file_size      INT UNSIGNED NOT NULL,   -- bytes
  uploaded_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_stored_name (stored_name),
  UNIQUE KEY uq_student_filetype (student_id, file_type),
  CONSTRAINT fk_files_student
    FOREIGN KEY (student_id) REFERENCES students(id)
    ON DELETE CASCADE
) ENGINE=InnoDB;
```

5. Algorithm / Procedure

1. Start. Import `sql/database.sql` to create the database and the two tables.
2. `config/db.php` opens the MySQLi connection (exceptions enabled) and starts the session.
3. **Add:** show the form → on POST verify the CSRF token → validate text fields and uniqueness → validate and move the uploaded files → insert the student and file records inside one transaction → redirect to the details page.
4. **View:** `students.php` selects all students (with optional search) using a prepared statement and joins the file table to show thumbnails and download links; `index.php` shows counts and recent students.
5. **Details:** `view_student.php` loads one student and its files and displays them; images are displayed through `download.php?inline=1`.
6. **Edit:** load the record into the same form → validate → update the row → if a new file is uploaded, replace the old file record and delete the old file from disk.
7. **Delete:** accept only a POST request with a valid token → read file names → delete the student (cascade removes file rows) → delete the files from disk.
8. **Download:** `download.php` looks the file up by id, validates the path with `realpath()` and streams it with safe headers.
9. On any database error: roll back the transaction, remove files already moved to disk, log the error and show a friendly message. Stop.

6. Project Structure

question2/

```
question2/
  index.php               dashboard
  students.php            student list + search
  add_student.php         add form + save
  edit_student.php        edit form + update
  delete_student.php      delete (POST + CSRF)
  view_student.php        student details page
  download.php            display / download uploaded files
  config/db.php           database connection + settings
  includes/               header.php, footer.php, student_form.php, functions.php
  assets/css/style.css    styling
  assets/js/app.js        client-side validation + delete confirmation
  sql/database.sql        database creation script
  uploads/                images/ and documents/ (blocked by .htaccess)
```

7. PHP Database Connection

question2/config/db.php

```
<?php
/**
 * config/db.php - database connection and global settings.
 * Change the credentials below to match your MySQL server.
 */

define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DB_NAME', 'student_management');

// Upload settings
define('UPLOAD_DIR', dirname(__DIR__) . '/uploads');   // private folder
define('MAX_IMAGE_SIZE', 2 * 1024 * 1024);              // 2 MB
define('MAX_DOC_SIZE', 5 * 1024 * 1024);                // 5 MB

// Make mysqli throw exceptions so errors can be handled with try/catch
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
    $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    $conn->set_charset('utf8mb4');
} catch (mysqli_sql_exception $e) {
    // Log the real error, show a generic message to the user
    error_log('DB connection failed: ' . $e->getMessage());
    http_response_code(500);
    die('Database connection failed. Please check config/db.php and make sure MySQL is running.');
}

if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
```

8. Helper Functions: Validation, CSRF and Secure File Upload

question2/includes/functions.php

```
<?php
/**
 * includes/functions.php - helpers: escaping, CSRF, flash messages,
 * validation and secure file upload handling.
 */

// ---------- Output escaping (prevents XSS) ----------
function e($value): string
{
    return htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8');
}

// ---------- CSRF protection ----------
function csrf_token(): string
{
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf'];
}

function csrf_field(): string
{
    return '<input type="hidden" name="csrf" value="' . e(csrf_token()) . '">';
}

function csrf_check(): void
{
    $sent = $_POST['csrf'] ?? '';
    if (!hash_equals($_SESSION['csrf'] ?? '', $sent)) {
        http_response_code(403);
        die('Invalid or missing security token. Please go back and try again.');
    }
}
```

question2/includes/functions.php (continued)

```
// ---------- Flash messages (shown once after a redirect) ----------
function flash(string $type, string $message): void
{
    $_SESSION['flash'][] = ['type' => $type, 'message' => $message];
}

function show_flash(): void
{
    if (!empty($_SESSION['flash'])) {
        foreach ($_SESSION['flash'] as $f) {
            echo '<div class="alert alert-' . e($f['type']) . '">' . e($f['message']) . '</div>';
        }
        unset($_SESSION['flash']);
    }
}

function redirect(string $url): void
{
    header('Location: ' . $url);
    exit;
}

// ---------- Course list used by the forms ----------
function course_options(): array
{
    return [
        'B.E. Computer Science and Engineering',
        'B.Tech Information Technology',
        'B.E. Electronics and Communication',
        'B.E. Electrical and Electronics',
        'B.E. Mechanical Engineering',
    ];
}

// ---------- Validation of text fields ----------
// Returns array of error messages (empty array = valid)
function validate_student(array $data, mysqli $conn, int $ignoreId = 0): array
{
    $errors = [];

    $name   = trim($data['student_name'] ?? '');
    $roll   = trim($data['roll_number'] ?? '');
    $email  = trim($data['email'] ?? '');
    $phone  = trim($data['phone'] ?? '');
    $course = trim($data['course'] ?? '');

    if ($name === '' || !preg_match("/^[A-Za-z][A-Za-z .'-]{2,99}$/", $name)) {
        $errors[] = 'Enter a valid student name (3-100 letters).';
    }
    if (!preg_match('/^[0-9]{2}[A-Za-z]{2,4}[0-9]{1,4}$/', $roll)) {
        $errors[] = 'Roll number format is invalid (example: 24CS255).';
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($email) > 120) {
        $errors[] = 'Enter a valid email address.';
    }
    if ($phone !== '' && !preg_match('/^[0-9+ -]{7,15}$/', $phone)) {
        $errors[] = 'Phone number must be 7-15 digits (+, space and - allowed).';
    }
    if (!in_array($course, course_options(), true)) {
        $errors[] = 'Please select a valid course.';
    }

    // Uniqueness checks with prepared statements
    if (!$errors) {
        $stmt = $conn->prepare('SELECT id FROM students WHERE roll_number = ? AND id <> ?');
        $stmt->bind_param('si', $roll, $ignoreId);
        $stmt->execute();
        if ($stmt->get_result()->num_rows > 0) {
            $errors[] = 'This roll number is already registered.';
        }
        $stmt->close();
```

question2/includes/functions.php (continued)

```
        $stmt = $conn->prepare('SELECT id FROM students WHERE email = ? AND id <> ?');
        $stmt->bind_param('si', $email, $ignoreId);
        $stmt->execute();
        if ($stmt->get_result()->num_rows > 0) {
            $errors[] = 'This email is already registered.';
        }
        $stmt->close();
    }
    return $errors;
}

// ---------- Secure file upload ----------
/**
 * Validates and moves one uploaded file.
 *
 * @param string $field  name of the <input type="file">
 * @param string $kind   'image' or 'document'
 * @param array  $errors collects error messages (by reference)
 * @return array|null    file info on success, null if nothing uploaded / error
 */
function process_upload(string $field, string $kind, array &$errors): ?array
{
    if (!isset($_FILES[$field]) || $_FILES[$field]['error'] === UPLOAD_ERR_NO_FILE) {
        return null;   // the file is optional
    }
    $file  = $_FILES[$field];
    $label = $kind === 'image' ? 'Profile image' : 'Document';

    // 1. PHP upload error codes
    $phpErrors = [
        UPLOAD_ERR_INI_SIZE   => 'exceeds the server upload limit.',
        UPLOAD_ERR_FORM_SIZE  => 'exceeds the form size limit.',
        UPLOAD_ERR_PARTIAL    => 'was only partially uploaded.',
        UPLOAD_ERR_NO_TMP_DIR => 'could not be saved (missing temp folder).',
        UPLOAD_ERR_CANT_WRITE => 'could not be written to disk.',
        UPLOAD_ERR_EXTENSION  => 'was blocked by a PHP extension.',
    ];
    if ($file['error'] !== UPLOAD_ERR_OK) {
        $errors[] = $label . ' ' . ($phpErrors[$file['error']] ?? 'upload failed.');
        return null;
    }
    if (!is_uploaded_file($file['tmp_name'])) {
        $errors[] = $label . ' upload was not valid.';
        return null;
    }

    // 2. Rules for each kind
    if ($kind === 'image') {
        $allowed = ['jpg' => ['image/jpeg'], 'jpeg' => ['image/jpeg'], 'png' => ['image/png']];
        $maxSize = MAX_IMAGE_SIZE;
    } else {
        $allowed = ['pdf' => ['application/pdf']];
        $maxSize = MAX_DOC_SIZE;
    }

    // 3. Size check
    if ($file['size'] <= 0 || $file['size'] > $maxSize) {
        $errors[] = $label . ' must be smaller than ' . ($maxSize / 1024 / 1024) . ' MB.';
        return null;
    }

    // 4. Extension check (case-insensitive whitelist)
    $ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
    if (!isset($allowed[$ext])) {
        $errors[] = $label . ' type not allowed. Allowed: ' . strtoupper(implode(', ',
            array_keys($allowed))) . '.';
        return null;
    }
```

question2/includes/functions.php (continued)

```
    // 5. Real MIME type check (does not trust the browser-supplied type)
    $finfo = new finfo(FILEINFO_MIME_TYPE);
    $mime  = $finfo->file($file['tmp_name']);
    if (!in_array($mime, $allowed[$ext], true)) {
        $errors[] = $label . ' content does not match its extension.';
        return null;
    }
    // Extra check: images must be decodable
    if ($kind === 'image' && @getimagesize($file['tmp_name']) === false) {
```

question2/includes/functions.php (continued)

```
        $errors[] = $label . ' is not a valid image.';
        return null;
    }

    // 6. Unique, unpredictable filename; original name is never used on disk
    $storedName = bin2hex(random_bytes(16)) . '.' . $ext;
    $subDir     = $kind === 'image' ? 'images' : 'documents';
    $targetDir  = UPLOAD_DIR . '/' . $subDir;
    if (!is_dir($targetDir) && !mkdir($targetDir, 0755, true)) {
        $errors[] = 'Upload folder could not be created.';
        return null;
    }
    if (!move_uploaded_file($file['tmp_name'], $targetDir . '/' . $storedName)) {
        $errors[] = $label . ' could not be saved.';
        return null;
    }

    // Original name: strip path parts and control characters
    $original = preg_replace('/[^\w .()-]/u', '_', basename($file['name']));

    return [
        'file_type'     => $kind,
        'original_name' => mb_substr($original, 0, 255),
        'stored_name'   => $storedName,
        'mime_type'     => $mime,
        'file_size'     => (int)$file['size'],
    ];
}

// Absolute path of a stored file ('' if unsafe / missing)
function stored_path(string $kind, string $storedName): string
{
    if (!preg_match('/^[a-f0-9]{32}\.(jpg|jpeg|png|pdf)$/', $storedName)) {
        return '';
    }
    $sub  = $kind === 'image' ? 'images' : 'documents';
    $path = realpath(UPLOAD_DIR . '/' . $sub . '/' . $storedName);
    $base = realpath(UPLOAD_DIR . '/' . $sub);
    if ($path === false || $base === false || strpos($path, $base . DIRECTORY_SEPARATOR) !== 0) {
        return '';
    }
    return $path;
}

function delete_stored_file(string $kind, string $storedName): void
{
    $path = stored_path($kind, $storedName);
    if ($path !== '' && is_file($path)) {
        @unlink($path);
    }
}

// Inserts a file metadata row (replaces an existing file of the same type)
function save_file_record(mysqli $conn, int $studentId, array $info): void
{
    // Find old file (if any) so it can be removed from disk after the swap
    $stmt = $conn->prepare('SELECT stored_name FROM student_files WHERE student_id = ? AND file_type = ?');
    $stmt->bind_param('is', $studentId, $info['file_type']);
    $stmt->execute();
    $old = $stmt->get_result()->fetch_assoc();
    $stmt->close();

    $stmt = $conn->prepare('DELETE FROM student_files WHERE student_id = ? AND file_type = ?');
    $stmt->bind_param('is', $studentId, $info['file_type']);
    $stmt->execute();
    $stmt->close();

    $stmt = $conn->prepare(
        'INSERT INTO student_files (student_id, file_type, original_name, stored_name, mime_type, file_size)
         VALUES (?, ?, ?, ?, ?, ?)'
    );
    $stmt->bind_param('issssi', $studentId, $info['file_type'], $info['original_name'],
                      $info['stored_name'], $info['mime_type'], $info['file_size']);
    $stmt->execute();
    $stmt->close();

    if ($old) {
        delete_stored_file($info['file_type'], $old['stored_name']);
    }
}
```

question2/includes/functions.php (continued)

```
function human_size(int $bytes): string
{
    if ($bytes >= 1048576) return round($bytes / 1048576, 2) . ' MB';
    if ($bytes >= 1024)    return round($bytes / 1024, 1) . ' KB';
    return $bytes . ' B';
}
```

9. Common Layout and Shared Form

question2/includes/header.php

```
<?php
/** includes/header.php - common page header and navigation */
require_once __DIR__ . '/../config/db.php';
require_once __DIR__ . '/functions.php';
$pageTitle = $pageTitle ?? 'Student Management System';
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?= e($pageTitle) ?> | SMS</title>
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
<header class="topbar">
  <div class="wrap">
    <span class="brand">Student Management System</span>
    <nav>
      <a href="index.php">Dashboard</a>
      <a href="students.php">Students</a>
      <a href="add_student.php">Add Student</a>
    </nav>
  </div>
</header>
<main class="wrap">
<?php show_flash(); ?>
```

question2/includes/footer.php

```
</main>
<footer class="footer">
  <div class="wrap">KPR Institute of Engineering and Technology &middot; Web Technology (U21CS501)</div>
</footer>
<script src="assets/js/app.js"></script>
</body>
</html>
```

question2/includes/student_form.php

```
<?php
/**
 * includes/student_form.php - form shared by add_student.php and edit_student.php
 * Expects: $student (array of current values), $isEdit (bool), $files (array)
 */
?>
<form method="post" enctype="multipart/form-data" id="studentForm" novalidate>
  <?= csrf_field() ?>

  <div class="grid">
    <div class="form-group">
      <label for="student_name">Student Name *</label>
      <input type="text" id="student_name" name="student_name" maxlength="100"
             value="<?= e($student['student_name'] ?? '') ?>" required>
    </div>

    <div class="form-group">
      <label for="roll_number">Roll Number *</label>
      <input type="text" id="roll_number" name="roll_number" maxlength="20"
             placeholder="24CS255" value="<?= e($student['roll_number'] ?? '') ?>" required>
    </div>
```

question2/includes/student_form.php (continued)

```
    <div class="form-group">
      <label for="email">Email *</label>
      <input type="email" id="email" name="email" maxlength="120"
             value="<?= e($student['email'] ?? '') ?>" required>
    </div>

    <div class="form-group">
      <label for="phone">Phone</label>
      <input type="text" id="phone" name="phone" maxlength="15"
             value="<?= e($student['phone'] ?? '') ?>">
    </div>

    <div class="form-group full">
      <label for="course">Course *</label>
      <select id="course" name="course" required>
        <option value="">-- Select Course --</option>
        <?php foreach (course_options() as $c): ?>
          <option value="<?= e($c) ?>" <?= (($student['course'] ?? '') === $c) ? 'selected' : '' ?>>
            <?= e($c) ?>
          </option>
        <?php endforeach; ?>
      </select>
    </div>

    <div class="form-group">
      <label for="profile_image">Profile Image (JPG/PNG, max 2 MB)</label>
      <input type="file" id="profile_image" name="profile_image" accept=".jpg,.jpeg,.png">
      <?php if ($isEdit && !empty($files['image'])): ?>
        <small>Current: <?= e($files['image']['original_name']) ?> (upload a new one to replace it)</small>
      <?php endif; ?>
    </div>

    <div class="form-group">
      <label for="document">Document (PDF, max 5 MB)</label>
      <input type="file" id="document" name="document" accept=".pdf">
      <?php if ($isEdit && !empty($files['document'])): ?>
        <small>Current: <?= e($files['document']['original_name']) ?> (upload a new one to replace
            it)</small>
      <?php endif; ?>
    </div>
  </div>

  <div class="actions">
    <button type="submit" class="btn btn-primary"><?= $isEdit ? 'Update Student' : 'Add Student' ?></button>
    <a href="students.php" class="btn btn-light">Cancel</a>
  </div>
</form>
```

10. Add Student

question2/add_student.php

```
<?php
/** add_student.php - create a new student with optional image and document */
$pageTitle = 'Add Student';
require_once __DIR__ . '/includes/header.php';

$errors  = [];
$student = [];
$files   = [];
$isEdit  = false;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $student = [
        'student_name' => trim($_POST['student_name'] ?? ''),
        'roll_number'  => strtoupper(trim($_POST['roll_number'] ?? '')),
        'email'        => trim($_POST['email'] ?? ''),
        'phone'        => trim($_POST['phone'] ?? ''),
        'course'       => trim($_POST['course'] ?? ''),
    ];

    // 1. Validate text fields
    $errors = validate_student($student, $conn);
```

question2/add_student.php (continued)

```
    // 2. Validate and move uploaded files (only if the text fields are fine)
    $image = $doc = null;
    if (!$errors) {
        $image = process_upload('profile_image', 'image', $errors);
        $doc   = process_upload('document', 'document', $errors);
    }

    // 3. Save everything in one transaction
    if (!$errors) {
        try {
            $conn->begin_transaction();

            $phone = $student['phone'] === '' ? null : $student['phone'];
            $stmt = $conn->prepare(
                'INSERT INTO students (student_name, roll_number, email, phone, course) VALUES (?, ?, ?, ?,
                    ?)'
            );
            $stmt->bind_param('sssss', $student['student_name'], $student['roll_number'],
                              $student['email'], $phone, $student['course']);
            $stmt->execute();
            $newId = $conn->insert_id;
            $stmt->close();

            foreach ([$image, $doc] as $info) {
                if ($info) save_file_record($conn, $newId, $info);
            }
            $conn->commit();

            flash('success', 'Student added successfully.');
            redirect('view_student.php?id=' . $newId);
        } catch (mysqli_sql_exception $ex) {
            $conn->rollback();
            // Remove files already moved to disk because the database save failed
            foreach ([$image, $doc] as $info) {
                if ($info) delete_stored_file($info['file_type'], $info['stored_name']);
            }
            error_log($ex->getMessage());
            $errors[] = 'Database error: the student could not be saved.';
        }
    } else {
        // Validation failed after files were moved: clean them up
        foreach ([$image ?? null, $doc ?? null] as $info) {
            if ($info) delete_stored_file($info['file_type'], $info['stored_name']);
        }
    }
}
?>
<h1>Add Student</h1>
<?php foreach ($errors as $err): ?>
  <div class="alert alert-error"><?= e($err) ?></div>
<?php endforeach; ?>
<div class="card">
  <?php require __DIR__ . '/includes/student_form.php'; ?>
</div>
<?php require __DIR__ . '/includes/footer.php'; ?>
```

11. View Students (Dashboard and List)

question2/index.php

```
<?php
/** index.php - dashboard with summary statistics and recent students */
$pageTitle = 'Dashboard';
require_once __DIR__ . '/includes/header.php';

try {
    // Summary counts (no user input, so a plain query is safe here)
    $totalStudents = (int)$conn->query('SELECT COUNT(*) FROM students')->fetch_row()[0];
    $totalImages   = (int)$conn->query("SELECT COUNT(*) FROM student_files WHERE
        file_type='image'")->fetch_row()[0];
    $totalDocs     = (int)$conn->query("SELECT COUNT(*) FROM student_files WHERE
        file_type='document'")->fetch_row()[0];
```

question2/index.php (continued)

```
    // Five most recently added students with their profile image id (if any)
    $recent = $conn->query(
        "SELECT s.id, s.student_name, s.roll_number, s.course, s.created_at, f.id AS image_id
           FROM students s
           LEFT JOIN student_files f ON f.student_id = s.id AND f.file_type = 'image'
          ORDER BY s.id DESC LIMIT 5"
    )->fetch_all(MYSQLI_ASSOC);
} catch (mysqli_sql_exception $ex) {
    error_log($ex->getMessage());
    echo '<div class="alert alert-error">Could not load dashboard data. Did you import
        sql/database.sql?</div>';
    require __DIR__ . '/includes/footer.php';
    exit;
}
?>
<h1>Dashboard</h1>

<div class="cards">
  <div class="card stat"><span class="num"><?= $totalStudents ?></span><span>Total Students</span></div>
  <div class="card stat"><span class="num"><?= $totalImages ?></span><span>Profile Images</span></div>
  <div class="card stat"><span class="num"><?= $totalDocs ?></span><span>Documents</span></div>
</div>

<div class="card">
  <h2>Recently Added Students</h2>
  <?php if (!$recent): ?>
    <p>No students yet. <a href="add_student.php">Add the first student</a>.</p>
  <?php else: ?>
  <table>
    <thead><tr><th>Photo</th><th>Name</th><th>Roll No.</th><th>Course</th><th>Added</th></tr></thead>
    <tbody>
    <?php foreach ($recent as $r): ?>
      <tr>
        <td>
          <?php if ($r['image_id']): ?>
            <img class="thumb" src="download.php?id=<?= (int)$r['image_id'] ?>&amp;inline=1" alt="Photo of
                <?= e($r['student_name']) ?>">
          <?php else: ?><span class="thumb placeholder">N/A</span><?php endif; ?>
        </td>
        <td><a href="view_student.php?id=<?= (int)$r['id'] ?>"><?= e($r['student_name']) ?></a></td>
        <td><?= e($r['roll_number']) ?></td>
        <td><?= e($r['course']) ?></td>
        <td><?= e(date('d M Y', strtotime($r['created_at']))) ?></td>
      </tr>
    <?php endforeach; ?>
    </tbody>
  </table>
  <?php endif; ?>
</div>
<?php require __DIR__ . '/includes/footer.php'; ?>
```

question2/students.php

```
<?php
/** students.php - list of all students with search and file links */
$pageTitle = 'Students';
require_once __DIR__ . '/includes/header.php';

$search = trim($_GET['q'] ?? '');

try {
    // Prepared statement with LIKE search (parameters are never concatenated)
    $sql = "SELECT s.id, s.student_name, s.roll_number, s.email, s.course,
                   img.id AS image_id, doc.id AS doc_id
              FROM students s
              LEFT JOIN student_files img ON img.student_id = s.id AND img.file_type = 'image'
              LEFT JOIN student_files doc ON doc.student_id = s.id AND doc.file_type = 'document'
             WHERE s.student_name LIKE ? OR s.roll_number LIKE ? OR s.email LIKE ?
             ORDER BY s.student_name ASC";
    $like = '%' . $search . '%';
    $stmt = $conn->prepare($sql);
    $stmt->bind_param('sss', $like, $like, $like);
    $stmt->execute();
    $students = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
    $stmt->close();
} catch (mysqli_sql_exception $ex) {
    error_log($ex->getMessage());
    $students = [];
    echo '<div class="alert alert-error">Could not load students.</div>';
}
```

question2/students.php (continued)

```
?>
<div class="page-head">
  <h1>Students</h1>
  <a class="btn btn-primary" href="add_student.php">+ Add Student</a>
</div>

<form method="get" class="search">
  <input type="text" name="q" placeholder="Search by name, roll number or email" value="<?= e($search) ?>">
  <button class="btn btn-primary" type="submit">Search</button>
  <?php if ($search !== ''): ?><a class="btn btn-light" href="students.php">Clear</a><?php endif; ?>
</form>

<div class="card">
<?php if (!$students): ?>
  <p>No students found.</p>
<?php else: ?>
  <table>
    <thead>
      <tr><th>#</th><th>Photo</th><th>Name</th><th>Roll
          No.</th><th>Email</th><th>Course</th><th>Document</th><th>Actions</th></tr>
    </thead>
    <tbody>
    <?php foreach ($students as $i => $s): ?>
      <tr>
        <td><?= $i + 1 ?></td>
        <td>
          <?php if ($s['image_id']): ?>
            <img class="thumb" src="download.php?id=<?= (int)$s['image_id'] ?>&amp;inline=1" alt="Photo of
                <?= e($s['student_name']) ?>">
          <?php else: ?><span class="thumb placeholder">N/A</span><?php endif; ?>
        </td>
        <td><?= e($s['student_name']) ?></td>
        <td><?= e($s['roll_number']) ?></td>
        <td><?= e($s['email']) ?></td>
        <td><?= e($s['course']) ?></td>
        <td>
          <?php if ($s['doc_id']): ?>
            <a href="download.php?id=<?= (int)$s['doc_id'] ?>">Download</a>
          <?php else: ?>&mdash;<?php endif; ?>
        </td>
        <td class="row-actions">
          <a href="view_student.php?id=<?= (int)$s['id'] ?>">View</a>
          <a href="edit_student.php?id=<?= (int)$s['id'] ?>">Edit</a>
          <!-- Delete uses POST + CSRF token, never a plain link -->
          <form method="post" action="delete_student.php" class="inline"
                data-confirm="Delete <?= e($s['student_name']) ?> and all uploaded files?">
            <?= csrf_field() ?>
            <input type="hidden" name="id" value="<?= (int)$s['id'] ?>">
            <button type="submit" class="link danger">Delete</button>
          </form>
        </td>
      </tr>
    <?php endforeach; ?>
    </tbody>
  </table>
<?php endif; ?>
</div>
<?php require __DIR__ . '/includes/footer.php'; ?>
```

12. Edit Student

question2/edit_student.php

```
<?php
/** edit_student.php - update a student; uploads replace existing files */
$pageTitle = 'Edit Student';
require_once __DIR__ . '/includes/header.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT) ?: 0;
if ($id <= 0) {
    flash('error', 'Invalid student id.');
    redirect('students.php');
}
```

question2/edit_student.php (continued)

```
// Load the existing student (prepared statement)
$stmt = $conn->prepare('SELECT * FROM students WHERE id = ?');
$stmt->bind_param('i', $id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
$stmt->close();

if (!$student) {
    flash('error', 'Student not found.');
    redirect('students.php');
}

// Load current files, indexed by type ('image' / 'document')
$files = [];
$stmt = $conn->prepare('SELECT * FROM student_files WHERE student_id = ?');
$stmt->bind_param('i', $id);
$stmt->execute();
foreach ($stmt->get_result()->fetch_all(MYSQLI_ASSOC) as $f) {
    $files[$f['file_type']] = $f;
}
$stmt->close();

$errors = [];
$isEdit = true;

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    csrf_check();
    $student = array_merge($student, [
        'student_name' => trim($_POST['student_name'] ?? ''),
        'roll_number'  => strtoupper(trim($_POST['roll_number'] ?? '')),
        'email'        => trim($_POST['email'] ?? ''),
        'phone'        => trim($_POST['phone'] ?? ''),
        'course'       => trim($_POST['course'] ?? ''),
    ]);

    $errors = validate_student($student, $conn, $id);   // $id is ignored in the unique check

    $image = $doc = null;
    if (!$errors) {
        $image = process_upload('profile_image', 'image', $errors);
        $doc   = process_upload('document', 'document', $errors);
    }

    if (!$errors) {
        try {
            $conn->begin_transaction();

            $phone = $student['phone'] === '' ? null : $student['phone'];
            $stmt = $conn->prepare(
                'UPDATE students SET student_name=?, roll_number=?, email=?, phone=?, course=? WHERE id=?'
            );
            $stmt->bind_param('sssssi', $student['student_name'], $student['roll_number'],
                              $student['email'], $phone, $student['course'], $id);
            $stmt->execute();
            $stmt->close();

            // save_file_record() swaps the old file for the new one
            foreach ([$image, $doc] as $info) {
                if ($info) save_file_record($conn, $id, $info);
            }
            $conn->commit();

            flash('success', 'Student updated successfully.');
            redirect('view_student.php?id=' . $id);
        } catch (mysqli_sql_exception $ex) {
            $conn->rollback();
            foreach ([$image, $doc] as $info) {
                if ($info) delete_stored_file($info['file_type'], $info['stored_name']);
            }
            error_log($ex->getMessage());
            $errors[] = 'Database error: the student could not be updated.';
        }
    } else {
        foreach ([$image ?? null, $doc ?? null] as $info) {
            if ($info) delete_stored_file($info['file_type'], $info['stored_name']);
        }
    }
}
?>
<h1>Edit Student</h1>
<?php foreach ($errors as $err): ?>
```

question2/edit_student.php (continued)

```
  <div class="alert alert-error"><?= e($err) ?></div>
<?php endforeach; ?>
<div class="card">
  <?php require __DIR__ . '/includes/student_form.php'; ?>
</div>
<?php require __DIR__ . '/includes/footer.php'; ?>
```

13. Delete Student

question2/delete_student.php

```
<?php
/**
 * delete_student.php - deletes a student and their files.
 * Only accepts POST requests protected by a CSRF token.
 */
require_once __DIR__ . '/config/db.php';
require_once __DIR__ . '/includes/functions.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    die('Method not allowed.');
}
csrf_check();

$id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT) ?: 0;
if ($id <= 0) {
    flash('error', 'Invalid student id.');
    redirect('students.php');
}

try {
    // 1. Remember the stored filenames before the rows disappear
    $stmt = $conn->prepare('SELECT file_type, stored_name FROM student_files WHERE student_id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $files = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
    $stmt->close();

    // 2. Delete the student (ON DELETE CASCADE removes student_files rows)
    $stmt = $conn->prepare('DELETE FROM students WHERE id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $deleted = $stmt->affected_rows;
    $stmt->close();

    // 3. Remove the physical files only after the database delete succeeded
    if ($deleted > 0) {
        foreach ($files as $f) {
            delete_stored_file($f['file_type'], $f['stored_name']);
        }
        flash('success', 'Student deleted successfully.');
    } else {
        flash('error', 'Student not found.');
    }
} catch (mysqli_sql_exception $ex) {
    error_log($ex->getMessage());
    flash('error', 'Database error: the student could not be deleted.');
}
redirect('students.php');
```

14. Student Details Page

question2/view_student.php

```
<?php
/** view_student.php - details page with profile image and document download */
$pageTitle = 'Student Details';
require_once __DIR__ . '/includes/header.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT) ?: 0;
if ($id <= 0) {
    flash('error', 'Invalid student id.');
    redirect('students.php');
}

$stmt = $conn->prepare('SELECT * FROM students WHERE id = ?');
$stmt->bind_param('i', $id);
$stmt->execute();
$s = $stmt->get_result()->fetch_assoc();
$stmt->close();

if (!$s) {
    flash('error', 'Student not found.');
    redirect('students.php');
}

$files = [];
$stmt = $conn->prepare('SELECT * FROM student_files WHERE student_id = ?');
$stmt->bind_param('i', $id);
$stmt->execute();
foreach ($stmt->get_result()->fetch_all(MYSQLI_ASSOC) as $f) {
    $files[$f['file_type']] = $f;
}
$stmt->close();
?>
<div class="page-head">
  <h1>Student Details</h1>
  <div>
    <a class="btn btn-primary" href="edit_student.php?id=<?= (int)$s['id'] ?>">Edit</a>
    <a class="btn btn-light" href="students.php">Back to list</a>
  </div>
</div>

<div class="card profile">
  <div class="photo">
    <?php if (!empty($files['image'])): ?>
      <img src="download.php?id=<?= (int)$files['image']['id'] ?>&amp;inline=1"
           alt="Profile photo of <?= e($s['student_name']) ?>">
    <?php else: ?>
      <div class="no-photo">No profile image</div>
    <?php endif; ?>
  </div>

  <table class="details">
    <tr><th>Name</th><td><?= e($s['student_name']) ?></td></tr>
    <tr><th>Roll Number</th><td><?= e($s['roll_number']) ?></td></tr>
    <tr><th>Email</th><td><?= e($s['email']) ?></td></tr>
    <tr><th>Phone</th><td><?= $s['phone'] !== null ? e($s['phone']) : '&mdash;' ?></td></tr>
    <tr><th>Course</th><td><?= e($s['course']) ?></td></tr>
    <tr><th>Registered</th><td><?= e(date('d M Y, h:i A', strtotime($s['created_at']))) ?></td></tr>
    <tr>
      <th>Document</th>
      <td>
        <?php if (!empty($files['document'])): $d = $files['document']; ?>
          <?= e($d['original_name']) ?> (<?= e(human_size((int)$d['file_size'])) ?>)
           <a href="download.php?id=<?= (int)$d['id'] ?>">Download</a>
        <?php else: ?>No document uploaded<?php endif; ?>
      </td>
    </tr>
    <?php if (!empty($files['image'])): $im = $files['image']; ?>
    <tr>
      <th>Image file</th>
      <td>
        <?= e($im['original_name']) ?> (<?= e(human_size((int)$im['file_size'])) ?>)
         <a href="download.php?id=<?= (int)$im['id'] ?>">Download</a>
      </td>
    </tr>
    <?php endif; ?>
  </table>
</div>
<?php require __DIR__ . '/includes/footer.php'; ?>
```

15. Display and Download of Uploaded Files

question2/download.php

```
<?php
/**
 * download.php - streams an uploaded file.
 *   download.php?id=5              -> forces download
 *   download.php?id=5&inline=1     -> shows images inside <img> tags
 * Files live in a protected folder, so this script is the only way to read them.
 */
require_once __DIR__ . '/config/db.php';
require_once __DIR__ . '/includes/functions.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT) ?: 0;
if ($id <= 0) {
    http_response_code(400);
    die('Invalid file id.');
}

try {
    $stmt = $conn->prepare('SELECT file_type, original_name, stored_name, mime_type FROM student_files
        WHERE id = ?');
    $stmt->bind_param('i', $id);
    $stmt->execute();
    $file = $stmt->get_result()->fetch_assoc();
    $stmt->close();
} catch (mysqli_sql_exception $ex) {
    error_log($ex->getMessage());
    http_response_code(500);
    die('Database error.');
}

if (!$file) {
    http_response_code(404);
    die('File not found.');
}

// Path traversal safe: stored_path() validates the name and uses realpath()
$path = stored_path($file['file_type'], $file['stored_name']);
if ($path === '' || !is_file($path)) {
    http_response_code(404);
    die('File is missing on the server.');
}

// Only images may be displayed inline; everything else is downloaded
$inline = isset($_GET['inline']) && $file['file_type'] === 'image';
$disposition = $inline ? 'inline' : 'attachment';

// Clean filename for the Content-Disposition header
$downloadName = str_replace(['"', "\r", "\n"], '', $file['original_name']);

header('Content-Type: ' . $file['mime_type']);
header('Content-Length: ' . filesize($path));
header('Content-Disposition: ' . $disposition . '; filename="' . $downloadName . '"');
header('X-Content-Type-Options: nosniff');
header('Cache-Control: private, max-age=3600');
readfile($path);
exit;
```

question2/uploads/.htaccess

```
# Block all direct web access to uploaded files.
# Files are served only through download.php after a database check.
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
<IfModule !mod_authz_core.c>
    Order allow,deny
    Deny from all
</IfModule>

# Never execute scripts from this folder
php_flag engine off
Options -ExecCGI -Indexes
```

16. Stylesheet and JavaScript

question2/assets/css/style.css

```
/* assets/css/style.css - styling for the Student Management System */
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: #f1f5f9;
  color: #1f2937;
  line-height: 1.5;
}

.wrap { max-width: 1100px; margin: 0 auto; padding: 0 18px; }
main.wrap { padding-top: 22px; padding-bottom: 40px; min-height: 70vh; }

/* ----- Top bar ----- */
.topbar { background: #1e3a8a; color: #fff; }
.topbar .wrap { display: flex; justify-content: space-between; align-items: center; padding-top: 14px;
    padding-bottom: 14px; flex-wrap: wrap; gap: 10px; }
.brand { font-size: 1.2rem; font-weight: 700; }
.topbar nav a { color: #dbeafe; text-decoration: none; margin-left: 18px; font-weight: 600; }
.topbar nav a:hover { color: #fff; text-decoration: underline; }

h1 { margin: 0 0 16px; color: #1e3a8a; font-size: 1.6rem; }
h2 { margin: 0 0 12px; font-size: 1.15rem; }
a { color: #1d4ed8; }

.page-head { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap:
    10px; }

/* ----- Cards ----- */
.card { background: #fff; border-radius: 8px; padding: 20px; margin-bottom: 20px; box-shadow: 0 2px 10px
    rgba(15,23,42,.08); overflow-x: auto; }
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 16px;
    margin-bottom: 20px; }
.stat { text-align: center; margin: 0; border-top: 4px solid #2563eb; }
.stat .num { display: block; font-size: 2.2rem; font-weight: 700; color: #1e3a8a; }

/* ----- Tables ----- */
table { width: 100%; border-collapse: collapse; }
th, td { padding: 10px 12px; text-align: left; border-bottom: 1px solid #e5e7eb; vertical-align: middle; }
thead th { background: #eff6ff; color: #1e3a8a; font-size: .9rem; }
tbody tr:hover { background: #f8fafc; }
.row-actions a, .row-actions .link { margin-right: 10px; }

.thumb { width: 44px; height: 44px; border-radius: 50%; object-fit: cover; display: inline-block; border:
    1px solid #cbd5e1; }
.thumb.placeholder { background: #e2e8f0; color: #64748b; font-size: .7rem; text-align: center;
    line-height: 44px; }

/* ----- Forms ----- */
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px 20px; }
.form-group.full { grid-column: 1 / -1; }
label { display: block; font-weight: 600; margin-bottom: 5px; font-size: .92rem; }
input[type=text], input[type=email], input[type=file], select {
  width: 100%; padding: 10px 12px; border: 1.5px solid #cbd5e1; border-radius: 6px; font-size: .95rem;
      background: #f9fafb;
}
input:focus, select:focus { outline: none; border-color: #2563eb; box-shadow: 0 0 0 3px
    rgba(37,99,235,.18); }
input.invalid, select.invalid { border-color: #dc2626; background: #fef2f2; }
small { color: #6b7280; display: block; margin-top: 4px; }
.actions { margin-top: 22px; display: flex; gap: 12px; }

form.inline { display: inline; }
.search { display: flex; gap: 10px; margin-bottom: 18px; flex-wrap: wrap; }
.search input { flex: 1; min-width: 220px; }

/* ----- Buttons ----- */
.btn { display: inline-block; padding: 9px 18px; border: none; border-radius: 6px; font-weight: 600;
    font-size: .95rem; cursor: pointer; text-decoration: none; }
.btn-primary { background: #1e3a8a; color: #fff; }
.btn-primary:hover { background: #1d4ed8; }
.btn-light { background: #e5e7eb; color: #374151; }
.btn-light:hover { background: #d1d5db; }
button.link { background: none; border: none; padding: 0; font: inherit; color: #1d4ed8; cursor: pointer;
    text-decoration: underline; }
button.link.danger { color: #dc2626; }
```

question2/assets/css/style.css (continued)

```
/* ----- Alerts ----- */
.alert { padding: 11px 14px; border-radius: 6px; margin-bottom: 12px; font-weight: 600; }
.alert-success { background: #dcfce7; border: 1px solid #16a34a; color: #166534; }
.alert-error { background: #fee2e2; border: 1px solid #dc2626; color: #991b1b; }

/* ----- Profile page ----- */
.profile { display: flex; gap: 28px; flex-wrap: wrap; align-items: flex-start; }
.profile .photo img { width: 180px; height: 180px; object-fit: cover; border-radius: 8px; border: 2px solid
    #cbd5e1; }
.no-photo { width: 180px; height: 180px; background: #e2e8f0; color: #64748b; display: flex; align-items:
    center; justify-content: center; border-radius: 8px; text-align: center; }
.details { flex: 1; min-width: 260px; }
.details th { width: 140px; background: none; color: #475569; }

.footer { background: #0f172a; color: #94a3b8; text-align: center; padding: 14px 0; font-size: .85rem; }

@media (max-width: 640px) {
  .grid { grid-template-columns: 1fr; }
}
```

question2/assets/js/app.js

```
// assets/js/app.js - client-side helpers (server-side validation is still the final authority)
"use strict";

document.addEventListener("DOMContentLoaded", function () {

  // ---------- Confirm before deleting ----------
  document.querySelectorAll("form[data-confirm]").forEach(function (form) {
    form.addEventListener("submit", function (event) {
      if (!window.confirm(form.getAttribute("data-confirm"))) {
        event.preventDefault();
      }
    });
  });

  // ---------- Student form validation ----------
  const form = document.getElementById("studentForm");
  if (!form) return;

  const MAX_IMAGE = 2 * 1024 * 1024;   // 2 MB
  const MAX_DOC = 5 * 1024 * 1024;     // 5 MB

  function fail(input, message, messages) {
    input.classList.add("invalid");
    messages.push(message);
  }

  form.addEventListener("submit", function (event) {
    const messages = [];
    form.querySelectorAll(".invalid").forEach(function (el) { el.classList.remove("invalid"); });

    const name = form.student_name;
    const roll = form.roll_number;
    const email = form.email;
    const course = form.course;
    const image = form.profile_image.files[0];
    const doc = form.document.files[0];

    if (!/^[A-Za-z][A-Za-z .'-]{2,99}$/.test(name.value.trim())) fail(name, "Enter a valid name (3-100
        letters).", messages);
    if (!/^[0-9]{2}[A-Za-z]{2,4}[0-9]{1,4}$/.test(roll.value.trim())) fail(roll, "Roll number must look
        like 24CS255.", messages);
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(email.value.trim())) fail(email, "Enter a valid email
        address.", messages);
    if (form.phone.value.trim() !== "" && !/^[0-9+ -]{7,15}$/.test(form.phone.value.trim()))
        fail(form.phone, "Phone must be 7-15 digits.", messages);
    if (course.value === "") fail(course, "Please select a course.", messages);

    if (image) {
      if (!/\.(jpe?g|png)$/i.test(image.name)) fail(form.profile_image, "Profile image must be JPG or
          PNG.", messages);
      else if (image.size > MAX_IMAGE) fail(form.profile_image, "Profile image must be under 2 MB.",
          messages);
    }
    if (doc) {
      if (!/\.pdf$/i.test(doc.name)) fail(form.document, "Document must be a PDF.", messages);
      else if (doc.size > MAX_DOC) fail(form.document, "Document must be under 5 MB.", messages);
    }
```

question2/assets/js/app.js (continued)

```
    if (messages.length > 0) {
      event.preventDefault();
      window.alert(messages.join("\n"));
      const firstBad = form.querySelector(".invalid");
      if (firstBad) firstBad.focus();
    }
  });
});
```

17. File Validation and Error Handling

Protection	How it is implemented
PHP upload errors	`$_FILES[...]['error']` is checked against every `UPLOAD_ERR_*` code with a readable message.
File size	Images ≤ 2 MB, PDFs ≤ 5 MB (`MAX_IMAGE_SIZE`, `MAX_DOC_SIZE`).
Extension whitelist	Images: jpg, jpeg, png. Documents: pdf. Everything else is rejected.
Real MIME type	`finfo` reads the file content; the MIME must match the extension. Browser-supplied types are never trusted. Images must also pass `getimagesize()`.
Unique file names	`bin2hex(random_bytes(16))` + extension; the user's file name is never used on disk and is stored only as display text.
Private storage	`uploads/.htaccess` denies direct access and script execution; files are served only through `download.php`.
Path traversal	`stored_path()` accepts only names matching a strict regex and verifies the `realpath()` is inside the upload folder.
SQL injection	Every query that uses input is a MySQLi prepared statement with bound parameters.
XSS	All output goes through `e()` (`htmlspecialchars`).
CSRF	Session token in every POST form, checked with `hash_equals()`; deletion is POST-only.
Database errors	`mysqli_report` throws exceptions; they are caught, logged with `error_log()`, the transaction is rolled back, moved files are deleted and a generic message is shown.

18. Explanation of Important Code

• **config/db.php** – `mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT)` turns database failures into exceptions so that `try/catch` blocks can handle them cleanly.

• **validate_student()** – validates name, roll number, e-mail, phone and course with regular expressions / `filter_var`, then runs two prepared `SELECT` queries to guarantee the roll number and e-mail are unique (the `id <> ?` part lets an edit keep its own values).

• **process_upload()** – runs the upload checks in a fixed order (error code, size, extension, real MIME type, image decode) before `move_uploaded_file()` stores the file under a random name.

• **save_file_record()** – finds the previous file of the same type, replaces its metadata row and deletes the old file from disk, so editing never leaves orphan files.

• **add_student.php / edit_student.php** – wrap all database changes in `begin_transaction()` … `commit()`; on an exception `rollback()` is called and uploaded files are removed so the disk and the database stay consistent.

• **delete_student.php** – reads file names first, deletes the row (cascade removes the metadata) and only then deletes the physical files.

• **download.php** – gets the record by id, builds a safe path, sets `Content-Type`, `Content-Disposition` (`inline` for images, `attachment` for others) and `X-Content-Type-Options: nosniff`, then streams the file.

• **assets/js/app.js** – repeats the validation in the browser for faster feedback and asks for confirmation before deleting; the server-side checks remain the final authority.

19. Result

A PHP and MySQL Student Management System was developed. The application is written to add, list, view, edit and delete student records, to accept a profile image and a PDF document with validation, to display the image and offer the document for download through a protected script, and to handle validation and database errors with friendly messages.

20. Output Screenshots

Set up the project as described in `question2/README.md`, then paste your own screenshots in the boxes below.

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.1 – Dashboard (index.php)

Figure 2.1 – Dashboard (index.php)

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.2 – Student list with thumbnails and actions (students.php)

Figure 2.2 – Student list with thumbnails and actions (students.php)

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.3 – Add Student form (add_student.php)

Figure 2.3 – Add Student form (add_student.php)

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.4 – Edit Student form (edit_student.php)

Figure 2.4 – Edit Student form (edit_student.php)

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.5 – Student details page with uploaded profile image and document

Figure 2.5 – Student details page with uploaded profile image and document

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 2.6 – Uploaded files: image display and document download / validation error message

Figure 2.6 – Uploaded files: image display and document download / validation error message

Question 3 — Java Servlet Student Profile Portal (16 Marks)

1. Aim

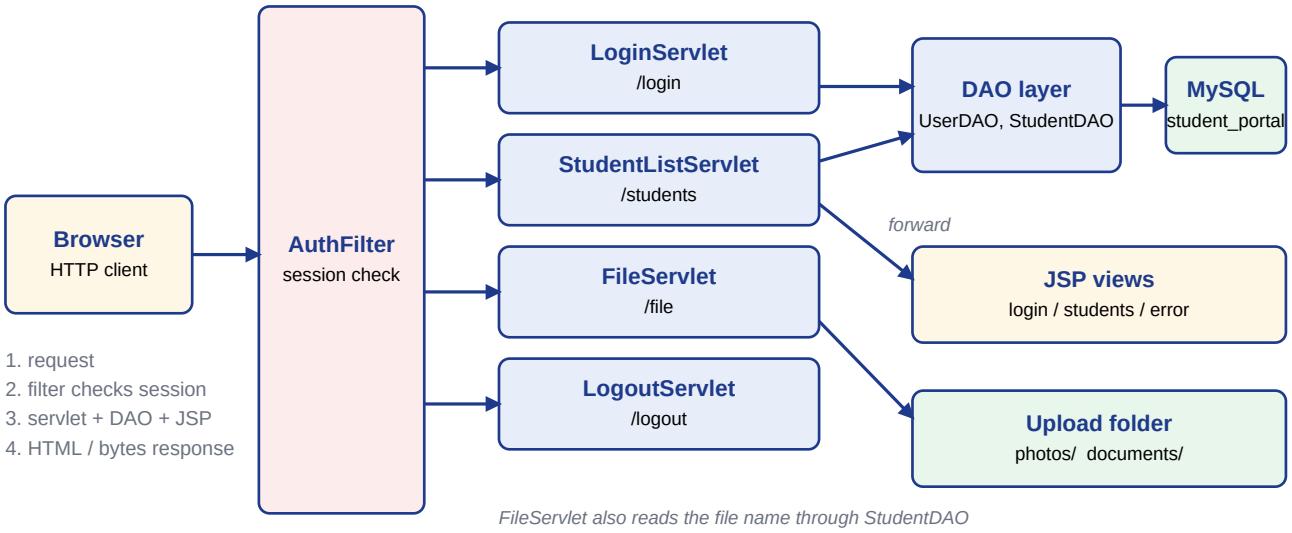
To develop a **Student Profile Portal** using Java Servlets, JSP, JDBC and MySQL in which an authenticated user logs in, views dynamically generated student profiles, and sees student photos and documents streamed by a servlet from a protected folder.

2. Software Requirements

Component	Requirement
JDK	Java 11 (Java 11-compatible source, `maven.compiler.release=11`)
Build tool	Apache Maven 3.6 or later
Server	Apache Tomcat 9.x (Servlet 4.0 / JSP 2.3, `javax.servlet` namespace)
Database	MySQL 8 (or MariaDB) with MySQL Connector/J 8.3.0
Libraries	JSTL 1.2 (packaged in the WAR), javax.servlet-api 4.0.1 and jsp-api 2.3.3 (provided by Tomcat)
IDE / Browser	IntelliJ IDEA / Eclipse / VS Code; Chrome, Edge or Firefox

3. System Architecture

The application follows the MVC pattern: servlets are controllers, JSP pages are views, and the model classes with DAOs access MySQL through JDBC. An authentication filter guards every protected URL.



Layer	Classes / files	Responsibility
Controller	LoginServlet, StudentListServlet, FileServlet, LogoutServlet	Handle HTTP requests and choose the response.
Filter	AuthFilter	Redirect anonymous users to /login; set no-cache headers.
Model	User, Student	Plain Java objects holding data.

Layer	Classes / files	Responsibility
DAO	UserDAO, StudentDAO	JDBC queries (prepared statements).
Utility	AppConfig, DBConnection, PasswordUtil	Configuration, connections, password hashing.
View	login.jsp, students.jsp, error.jsp	Generate HTML using JSTL / EL (kept in WEB-INF).

4. Database Design and SQL Script

Table	Columns	Notes
users	id, username (UNIQUE), full_name, password_hash, created_at	password_hash = `pbkdf2$iterations$salt$hash`
students	id, roll_number (UNIQUE), name, email (UNIQUE), course, photo_file, document_file, created_at	photo_file / document_file hold only a file name; files are stored on disk

question3/sql/schema.sql

```
-- schema.sql : Student Profile Portal (Question 3)
-- Run with:  mysql -u root -p < sql/schema.sql

CREATE DATABASE IF NOT EXISTS student_portal
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE student_portal;

-- ------------------------------------------------------
-- users: portal login accounts (passwords are stored hashed)
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS users (
  id             INT UNSIGNED NOT NULL AUTO_INCREMENT,
  username       VARCHAR(50)  NOT NULL,
  full_name      VARCHAR(100) NOT NULL,
  password_hash  VARCHAR(255) NOT NULL,   -- pbkdf2$iterations$saltHex$hashHex
  created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_username (username)
) ENGINE=InnoDB;

-- ------------------------------------------------------
-- students: profile data. photo_file / document_file hold only a
-- file NAME; the file lives in <upload.dir>/photos or /documents.
-- ------------------------------------------------------
CREATE TABLE IF NOT EXISTS students (
  id             INT UNSIGNED NOT NULL AUTO_INCREMENT,
  roll_number    VARCHAR(20)  NOT NULL,
  name           VARCHAR(100) NOT NULL,
  email          VARCHAR(120) NOT NULL,
  course         VARCHAR(80)  NOT NULL,
  photo_file     VARCHAR(100) DEFAULT NULL,
  document_file  VARCHAR(100) DEFAULT NULL,
  created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_students_roll (roll_number),
  UNIQUE KEY uq_students_email (email)
) ENGINE=InnoDB;

-- ------------------------------------------------------
-- Sample login:  username = admin   password = Admin@123
-- (hash generated with com.kpr.portal.util.PasswordUtil)
-- Change this password for any real use.
-- ------------------------------------------------------
INSERT INTO users (username, full_name, password_hash) VALUES
('admin', 'Portal Administrator',
 'pbkdf2$120000$a34be83e19f28dd598426968b90f7005$0be6c1d40fdecb5f05826775c7ea94f12a93691f9659f0fbd309b833d8a
     1d67d');
```

question3/sql/schema.sql (continued)

```
-- ------------------------------------------------------
-- Sample students (no files yet - see README for attaching your own
-- photos / PDFs with the UPDATE statements below)
-- ------------------------------------------------------
INSERT INTO students (roll_number, name, email, course) VALUES
('24CS255', 'Amar Kumar Pandit', 'amar.pandit@example.com',  'B.E. Computer Science and Engineering'),
('24CS101', 'Priya Sharma',      'priya.sharma@example.com', 'B.E. Computer Science and Engineering'),
('24IT042', 'Rahul Verma',       'rahul.verma@example.com',  'B.Tech Information Technology');

-- To attach files: copy them into <upload.dir>/photos and <upload.dir>/documents, then e.g.
--   UPDATE students SET photo_file = 'amar.jpg', document_file = 'amar_resume.pdf'
--    WHERE roll_number = '24CS255';
-- File names may contain only letters, digits, dot, underscore, hyphen and must
-- end in .jpg, .jpeg, .png (photos) or .pdf (documents).
```

5. Algorithm / Procedure

1. Start. Create the database with `schema.sql`; configure `db.properties`; create the upload folders `photos` and `documents`.
2. The browser requests `/students`. `AuthFilter` checks `session.getAttribute("user")`; if it is missing the user is redirected to `/login`.
3. `LoginServlet` (GET) creates a CSRF token and forwards to `login.jsp`.
4. `LoginServlet` (POST) checks the token and required fields, loads the user with `UserDAO` (prepared statement) and verifies the password with `PasswordUtil.verify()`.
5. On failure, show a generic message (HTTP 401). On success, invalidate the old session, create a new one, store the `User` (without hash), set a 30-minute timeout and redirect to `/students`.
6. `StudentListServlet` calls `StudentDAO.findAll(search)`, stores the list in a request attribute and forwards to `students.jsp`.
7. `students.jsp` loops over the list with `<c:forEach>`; `<c:choose>` / `<c:if>` decide between a photo and an initial letter and whether to show document links.
8. For each photo the browser requests `/file?type=photo&id=N`. `FileServlet` reads the file name from the database, validates it, resolves the path inside the upload folder and streams the bytes.
9. `LogoutServlet` (POST) invalidates the session and redirects to the login page. Stop.

6. pom.xml and Required Configuration

question3/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.kpr.portal</groupId>
  <artifactId>student-portal</artifactId>
  <version>1.0.0</version>
  <packaging>war</packaging>
  <name>Student Profile Portal</name>

  <properties>
    <maven.compiler.release>11</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <failOnMissingWebXml>false</failOnMissingWebXml>
  </properties>
```

question3/pom.xml (continued)

```
  <dependencies>
    <!-- Servlet and JSP APIs: provided by Apache Tomcat 9 (javax.* namespace) -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>

    <!-- JSTL (Tomcat does not ship it, so it is packaged inside the WAR) -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
      <version>1.2</version>
    </dependency>

    <!-- MySQL JDBC driver -->
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.3.0</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>student-portal</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

question3/src/main/webapp/WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
             http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
         version="4.0">

  <display-name>Student Profile Portal</display-name>

  <!-- Servlets and the filter are registered with annotations (@WebServlet, @WebFilter) -->

  <!-- Session settings: 30 minutes, cookie not readable from JavaScript -->
  <session-config>
    <session-timeout>30</session-timeout>
    <cookie-config>
      <http-only>true</http-only>
    </cookie-config>
    <tracking-mode>COOKIE</tracking-mode>
  </session-config>

  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <!-- Friendly error pages -->
  <error-page>
    <error-code>404</error-code>
    <location>/WEB-INF/views/error.jsp</location>
  </error-page>
  <error-page>
    <error-code>500</error-code>
    <location>/WEB-INF/views/error.jsp</location>
  </error-page>
</web-app>
```

question3/src/main/resources/db.properties

```
# db.properties - database and file-storage settings (edit before deploying)
db.url=jdbc:mysql://localhost:3306/student_portal?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=U
    TC&characterEncoding=UTF-8
db.user=root
db.password=

# Folder that contains the sub-folders "photos" and "documents".
# ${user.home} is replaced with the home directory of the Tomcat user.
upload.dir=${user.home}/student-portal-uploads
```

Project structure

question3/

```
question3/
  pom.xml
  sql/schema.sql
  src/main/java/com/kpr/portal/
    util/     AppConfig.java, DBConnection.java, PasswordUtil.java
    model/    User.java, Student.java
    dao/      UserDAO.java, StudentDAO.java
    filter/   AuthFilter.java
    servlet/  LoginServlet.java, StudentListServlet.java,
              FileServlet.java, LogoutServlet.java
  src/main/resources/db.properties
  src/main/webapp/
    index.jsp, css/style.css
    WEB-INF/web.xml
    WEB-INF/views/  login.jsp, students.jsp, error.jsp
```

7. JDBC Database Connection Class

question3/src/main/java/com/kpr/portal/util/AppConfig.java

```
package com.kpr.portal.util;

import java.io.IOException;
import java.io.InputStream;
import java.io.UncheckedIOException;
import java.util.Properties;

/**
 * Loads db.properties from the classpath once and exposes the values.
 */
public final class AppConfig {

    private static final Properties PROPS = new Properties();

    static {
        try (InputStream in = AppConfig.class.getClassLoader().getResourceAsStream("db.properties")) {
            if (in == null) {
                throw new IllegalStateException("db.properties not found on the classpath");
            }
            PROPS.load(in);
        } catch (IOException e) {
            throw new UncheckedIOException("Could not read db.properties", e);
        }
    }

    private AppConfig() { }

    /** Returns a property value, expanding ${user.home}. */
    public static String get(String key) {
        String value = PROPS.getProperty(key, "");
        return value.replace("${user.home}", System.getProperty("user.home"));
    }
}
```

question3/src/main/java/com/kpr/portal/util/DBConnection.java

```
package com.kpr.portal.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * JDBC connection helper. Every DAO method opens a connection with
 * getConnection() inside try-with-resources so it is always closed.
 */
public final class DBConnection {

    static {
        try {
            // Registers the MySQL driver (needed inside Tomcat's class loader)
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new ExceptionInInitializerError("MySQL JDBC driver not found: " + e.getMessage());
        }
    }

    private DBConnection() { }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
                AppConfig.get("db.url"),
                AppConfig.get("db.user"),
                AppConfig.get("db.password"));
    }
}
```

8. Password Hashing Utility

question3/src/main/java/com/kpr/portal/util/PasswordUtil.java

```
package com.kpr.portal.util;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

/**
 * Salted password hashing with PBKDF2-HMAC-SHA256 (built into the JDK).
 * Stored format:  pbkdf2$<iterations>$<saltHex>$<hashHex>
 *
 * To create a hash for the users table run (after "mvn compile"):
 *   java -cp target/classes com.kpr.portal.util.PasswordUtil MyPassword
 */
public final class PasswordUtil {

    private static final String ALGORITHM = "PBKDF2WithHmacSHA256";
    private static final int ITERATIONS = 120_000;
    private static final int SALT_BYTES = 16;
    private static final int KEY_BITS = 256;
    private static final SecureRandom RANDOM = new SecureRandom();

    private PasswordUtil() { }

    public static String hash(String password) {
        byte[] salt = new byte[SALT_BYTES];
        RANDOM.nextBytes(salt);
        byte[] hash = pbkdf2(password.toCharArray(), salt, ITERATIONS);
        return "pbkdf2$" + ITERATIONS + "$" + toHex(salt) + "$" + toHex(hash);
    }

    public static boolean verify(String password, String stored) {
        if (password == null || stored == null) {
            return false;
        }
        String[] parts = stored.split("\\$");
        if (parts.length != 4 || !"pbkdf2".equals(parts[0])) {
            return false;
        }
        try {
            int iterations = Integer.parseInt(parts[1]);
            byte[] salt = fromHex(parts[2]);
            byte[] expected = fromHex(parts[3]);
            byte[] actual = pbkdf2(password.toCharArray(), salt, iterations);
            // Constant-time comparison prevents timing attacks
            return MessageDigest.isEqual(expected, actual);
        } catch (IllegalArgumentException e) {
            return false;   // malformed stored hash
        }
    }

    private static byte[] pbkdf2(char[] password, byte[] salt, int iterations) {
        try {
            PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, KEY_BITS);
            return SecretKeyFactory.getInstance(ALGORITHM).generateSecret(spec).getEncoded();
        } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
            throw new IllegalStateException("Password hashing failed", e);
        }
    }

    private static String toHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder(bytes.length * 2);
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
```

question3/src/main/java/com/kpr/portal/util/PasswordUtil.java (continued)

```
    private static byte[] fromHex(String hex) {
        if (hex.length() % 2 != 0) {
            throw new IllegalArgumentException("odd length");
        }
        byte[] out = new byte[hex.length() / 2];
        for (int i = 0; i < out.length; i++) {
            out[i] = (byte) Integer.parseInt(hex.substring(2 * i, 2 * i + 2), 16);
        }
        return out;
    }

    /** Small command-line helper to generate hashes for the SQL script. */
    public static void main(String[] args) {
        if (args.length != 1) {
            System.out.println("Usage: java -cp target/classes com.kpr.portal.util.PasswordUtil
                <password>");
            return;
        }
        System.out.println(hash(args[0]));
    }
}
```

9. Student Model and DAO Classes

question3/src/main/java/com/kpr/portal/model/Student.java

```
package com.kpr.portal.model;

/** Student model (one row of the students table). */
public class Student {

    private int id;
    private String rollNumber;
    private String name;
    private String email;
    private String course;
    private String photoFile;      // file name inside <upload.dir>/photos (may be null)
    private String documentFile;   // file name inside <upload.dir>/documents (may be null)

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getRollNumber() { return rollNumber; }
    public void setRollNumber(String rollNumber) { this.rollNumber = rollNumber; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }

    public String getPhotoFile() { return photoFile; }
    public void setPhotoFile(String photoFile) { this.photoFile = photoFile; }

    public String getDocumentFile() { return documentFile; }
    public void setDocumentFile(String documentFile) { this.documentFile = documentFile; }

    // EL helpers used by the JSP: ${student.hasPhoto}, ${student.hasDocument}
    public boolean isHasPhoto() {
        return photoFile != null && !photoFile.isEmpty();
    }

    public boolean isHasDocument() {
        return documentFile != null && !documentFile.isEmpty();
    }

    /** First letter of the name, shown when there is no photo. */
    public String getInitial() {
        return (name == null || name.isEmpty()) ? "?" : name.substring(0, 1).toUpperCase();
    }
}
```

question3/src/main/java/com/kpr/portal/model/User.java

```
package com.kpr.portal.model;

/** A portal user (login account). */
public class User {

    private int id;
    private String username;
    private String fullName;
    private String passwordHash;

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }

    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
}
```

question3/src/main/java/com/kpr/portal/dao/StudentDAO.java

```
package com.kpr.portal.dao;

import com.kpr.portal.model.Student;
import com.kpr.portal.util.DBConnection;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/** Data access for the students table. All queries use prepared statements. */
public class StudentDAO {

    private static final String COLUMNS =
            "id, roll_number, name, email, course, photo_file, document_file";

    /** Returns all students, optionally filtered by name / roll number / email. */
    public List<Student> findAll(String search) throws SQLException {
        String sql = "SELECT " + COLUMNS + " FROM students "
                   + "WHERE name LIKE ? OR roll_number LIKE ? OR email LIKE ? "
                   + "ORDER BY name";
        String like = "%" + (search == null ? "" : search) + "%";

        List<Student> list = new ArrayList<>();
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, like);
            ps.setString(2, like);
            ps.setString(3, like);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    list.add(map(rs));
                }
            }
        }
        return list;
    }

    /** Returns one student by primary key, or null if not found. */
    public Student findById(int id) throws SQLException {
        String sql = "SELECT " + COLUMNS + " FROM students WHERE id = ?";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? map(rs) : null;
            }
        }
    }
```

question3/src/main/java/com/kpr/portal/dao/StudentDAO.java (continued)

```
    // Converts the current ResultSet row to a Student object
    private Student map(ResultSet rs) throws SQLException {
        Student s = new Student();
        s.setId(rs.getInt("id"));
        s.setRollNumber(rs.getString("roll_number"));
        s.setName(rs.getString("name"));
        s.setEmail(rs.getString("email"));
        s.setCourse(rs.getString("course"));
        s.setPhotoFile(rs.getString("photo_file"));
        s.setDocumentFile(rs.getString("document_file"));
        return s;
    }
}
```

question3/src/main/java/com/kpr/portal/dao/UserDAO.java

```
package com.kpr.portal.dao;

import com.kpr.portal.model.User;
import com.kpr.portal.util.DBConnection;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/** Data access for the users table. */
public class UserDAO {

    /** Returns the user with the given username, or null if none exists. */
    public User findByUsername(String username) throws SQLException {
        String sql = "SELECT id, username, full_name, password_hash FROM users WHERE username = ?";
        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, username);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    User u = new User();
                    u.setId(rs.getInt("id"));
                    u.setUsername(rs.getString("username"));
                    u.setFullName(rs.getString("full_name"));
                    u.setPasswordHash(rs.getString("password_hash"));
                    return u;
                }
                return null;
            }
        }
    }
}
```

10. Login Servlet and Session Handling

question3/src/main/java/com/kpr/portal/servlet/LoginServlet.java

```
package com.kpr.portal.servlet;

import com.kpr.portal.dao.UserDAO;
import com.kpr.portal.model.User;
import com.kpr.portal.util.PasswordUtil;
import java.io.IOException;
import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
```

question3/src/main/java/com/kpr/portal/servlet/LoginServlet.java (continued)

```
/**
 * Handles login: GET shows the form, POST checks the credentials.
 */
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private static final Logger LOG = Logger.getLogger(LoginServlet.class.getName());
    private static final String LOGIN_VIEW = "/WEB-INF/views/login.jsp";
    private static final SecureRandom RANDOM = new SecureRandom();
    private final UserDAO userDAO = new UserDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // Already logged in? go straight to the student list
        HttpSession existing = request.getSession(false);
        if (existing != null && existing.getAttribute("user") != null) {
            response.sendRedirect(request.getContextPath() + "/students");
            return;
        }

        // Create a CSRF token that must come back with the POST
        HttpSession session = request.getSession(true);
        session.setAttribute("loginToken", new BigInteger(130, RANDOM).toString(32));
        request.getRequestDispatcher(LOGIN_VIEW).forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        String username = trim(request.getParameter("username"));
        String password = request.getParameter("password");
        String sentToken = request.getParameter("token");

        // 1. CSRF token check
        HttpSession old = request.getSession(false);
        String expected = old == null ? null : (String) old.getAttribute("loginToken");
        if (expected == null || sentToken == null
                || !MessageDigest.isEqual(expected.getBytes(), sentToken.getBytes())) {
            response.sendError(HttpServletResponse.SC_FORBIDDEN, "Invalid form token. Reload the login
                page.");
            return;
        }

        // 2. Required fields
        if (username.isEmpty() || password == null || password.isEmpty()) {
            fail(request, response, "Username and password are required.", username);
            return;
        }

        try {
            // 3. Look the user up and verify the salted hash
            User user = userDAO.findByUsername(username);
            if (user == null || !PasswordUtil.verify(password, user.getPasswordHash())) {
                // Same message for both cases so attackers cannot learn which usernames exist
                fail(request, response, "Invalid username or password.", username);
                return;
            }

            // 4. Success: replace the old session (prevents session fixation)
            old.invalidate();
            HttpSession session = request.getSession(true);
            user.setPasswordHash(null);                 // never keep the hash in the session
            session.setAttribute("user", user);
            session.setMaxInactiveInterval(30 * 60);    // 30 minutes
            response.sendRedirect(request.getContextPath() + "/students");

        } catch (SQLException e) {
            LOG.log(Level.SEVERE, "Login query failed", e);
            fail(request, response, "Service unavailable. Please try again later.", username);
        }
    }
```

question3/src/main/java/com/kpr/portal/servlet/LoginServlet.java (continued)

```
    // Shows the login form again with an error message
    private void fail(HttpServletRequest request, HttpServletResponse response,
                      String message, String username) throws ServletException, IOException {
        request.setAttribute("error", message);
        request.setAttribute("username", username);
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        request.getRequestDispatcher(LOGIN_VIEW).forward(request, response);
    }

    private static String trim(String s) {
        return s == null ? "" : s.trim();
    }
}
```

11. Authentication Filter (Session Check)

question3/src/main/java/com/kpr/portal/filter/AuthFilter.java

```
package com.kpr.portal.filter;

import java.io.IOException;
import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * Authentication filter: every protected URL requires a logged-in session.
 * Anonymous users are redirected to the login page.
 */
@WebFilter(urlPatterns = {"/students", "/file", "/logout"})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        // false = do not create a new session just to check it
        HttpSession session = request.getSession(false);
        boolean loggedIn = session != null && session.getAttribute("user") != null;

        if (loggedIn) {
            // Pages that show personal data must not be cached by the browser
            response.setHeader("Cache-Control", "no-store, no-cache, must-revalidate");
            response.setHeader("Pragma", "no-cache");
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login");
        }
    }
}
```

12. Student List Servlet

question3/src/main/java/com/kpr/portal/servlet/StudentListServlet.java

```
package com.kpr.portal.servlet;

import com.kpr.portal.dao.StudentDAO;
import com.kpr.portal.model.Student;
import java.io.IOException;
import java.sql.SQLException;
import java.util.Collections;
import java.util.List;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * Loads students from the database and forwards them to students.jsp.
 * Access is protected by AuthFilter.
 */
@WebServlet("/students")
public class StudentListServlet extends HttpServlet {

    private static final Logger LOG = Logger.getLogger(StudentListServlet.class.getName());
    private final StudentDAO studentDAO = new StudentDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String search = request.getParameter("q");
        search = (search == null) ? "" : search.trim();
        if (search.length() > 50) {
            search = search.substring(0, 50);   // limit input length
        }

        List<Student> students;
        try {
            students = studentDAO.findAll(search);
        } catch (SQLException e) {
            LOG.log(Level.SEVERE, "Could not load students", e);
            students = Collections.emptyList();
            request.setAttribute("error", "Could not load students from the database.");
        }

        // These attributes are read by the JSP using Expression Language
        request.setAttribute("students", students);
        request.setAttribute("search", search);
        request.getRequestDispatcher("/WEB-INF/views/students.jsp").forward(request, response);
    }
}
```

13. JSP Pages: Dynamic Profiles, Looping and Conditional Logic

`students.jsp` uses `<c:forEach>` for the loop, `<c:choose>/<c:when>/<c:otherwise>` and `<c:if>` for the conditions, and `<c:out>` to escape every value. The `<img>` tag points to `FileServlet` instead of a static path.

question3/src/main/webapp/WEB-INF/views/students.jsp

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profiles | Student Profile Portal</title>
  <link rel="stylesheet" href="${ctx}/css/style.css">
</head>
<body>
<header class="topbar">
  <span class="brand">Student Profile Portal</span>
  <div class="user">
```

question3/src/main/webapp/WEB-INF/views/students.jsp (continued)

```
    Signed in as <strong><c:out value="${sessionScope.user.fullName}"/></strong>
    <form method="post" action="${ctx}/logout" class="inline">
      <button type="submit" class="btn btn-small">Logout</button>
    </form>
  </div>
</header>

<main>
  <h1>Student Profiles</h1>

  <form method="get" action="${ctx}/students" class="search">
    <input type="text" name="q" maxlength="50" placeholder="Search by name, roll number or email"
           value="<c:out value='${search}'/>">
    <button type="submit" class="btn">Search</button>
    <c:if test="${not empty search}"><a class="btn btn-light" href="${ctx}/students">Clear</a></c:if>
  </form>

  <c:if test="${not empty error}">
    <div class="alert alert-error"><c:out value="${error}"/></div>
  </c:if>

  <c:choose>
    <%-- Conditional: nothing to show --%>
    <c:when test="${empty students}">
      <p class="muted">No students found.</p>
    </c:when>

    <c:otherwise>
      <p class="muted">Showing ${students.size()} student(s).</p>
      <div class="grid">
        <%-- Loop: one profile card per student --%>
        <c:forEach var="s" items="${students}" varStatus="loop">
          <article class="card">
            <div class="photo">
              <c:choose>
                <c:when test="${s.hasPhoto}">
                  <%-- The image bytes come from FileServlet, not from a static folder --%>
                  <img src="${ctx}/file?type=photo&amp;id=${s.id}"
                       alt="Photo of <c:out value='${s.name}'/>">
                </c:when>
                <c:otherwise>
                  <div class="initial"><c:out value="${s.initial}"/></div>
                </c:otherwise>
              </c:choose>
            </div>

            <h2><c:out value="${loop.count}. ${s.name}"/></h2>
            <dl>
              <dt>Roll No.</dt><dd><c:out value="${s.rollNumber}"/></dd>
              <dt>Email</dt><dd><c:out value="${s.email}"/></dd>
              <dt>Course</dt><dd><c:out value="${s.course}"/></dd>
            </dl>

            <div class="links">
              <c:if test="${s.hasDocument}">
                <a href="${ctx}/file?type=doc&amp;id=${s.id}" target="_blank" rel="noopener">View
                    document</a>
                <a href="${ctx}/file?type=doc&amp;id=${s.id}&amp;download=1">Download</a>
              </c:if>
              <c:if test="${not s.hasDocument}">
                <span class="muted">No document</span>
              </c:if>
            </div>
          </article>
        </c:forEach>
      </div>
    </c:otherwise>
  </c:choose>
</main>
</body>
</html>
```

question3/src/main/webapp/WEB-INF/views/login.jsp

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login | Student Profile Portal</title>
  <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body class="login-page">
  <div class="login-box">
    <h1>Student Profile Portal</h1>
    <p class="muted">KPR Institute of Engineering and Technology</p>

    <%-- Conditional logic: show a message only when it exists --%>
    <c:if test="${not empty error}">
      <div class="alert alert-error"><c:out value="${error}"/></div>
    </c:if>
    <c:if test="${param.loggedout == '1' and empty error}">
      <div class="alert alert-success">You have been logged out.</div>
    </c:if>

    <form method="post" action="${pageContext.request.contextPath}/login" autocomplete="off">
      <input type="hidden" name="token" value="<c:out value='${sessionScope.loginToken}'/>">

      <label for="username">Username</label>
      <input type="text" id="username" name="username" maxlength="50" required
             value="<c:out value='${username}'/>">

      <label for="password">Password</label>
      <input type="password" id="password" name="password" maxlength="100" required>

      <button type="submit" class="btn">Login</button>
    </form>
  </div>
</body>
</html>
```

question3/src/main/webapp/WEB-INF/views/error.jsp

```
<%@ page contentType="text/html; charset=UTF-8" language="java" isErrorPage="true" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Error | Student Profile Portal</title>
  <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body class="login-page">
  <div class="login-box">
    <h1>Something went wrong</h1>
    <p class="muted">The page you requested could not be shown. No technical details are displayed for
        security reasons.</p>
    <a class="btn" href="${pageContext.request.contextPath}/students">Back to students</a>
  </div>
</body>
</html>
```

question3/src/main/webapp/index.jsp

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%-- Entry point: send the visitor to the student list (AuthFilter redirects to login if needed) --%>
<c:redirect url="/students"/>
```

14. Image and Document Streaming Servlet

question3/src/main/java/com/kpr/portal/servlet/FileServlet.java

```
package com.kpr.portal.servlet;

import com.kpr.portal.dao.StudentDAO;
import com.kpr.portal.model.Student;
import com.kpr.portal.util.AppConfig;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.sql.SQLException;
import java.util.Locale;
import java.util.logging.Level;
import java.util.logging.Logger;
import java.util.regex.Pattern;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * Streams a student's photo or document from the upload folder.
 *   /file?type=photo&id=3           -> image shown inside an <img> tag
 *   /file?type=doc&id=3             -> PDF opened in the browser
 *   /file?type=doc&id=3&download=1  -> PDF forced to download
 *
 * The browser only sends a student id; the file name always comes from
 * the database, and the final path is checked to stay inside the upload folder.
 */
@WebServlet("/file")
public class FileServlet extends HttpServlet {

    private static final Logger LOG = Logger.getLogger(FileServlet.class.getName());
    // Only simple file names with a known extension are ever accepted
    private static final Pattern SAFE_NAME =
        Pattern.compile("^[A-Za-z0-9][A-Za-z0-9._-]{0,99}\\.(jpg|jpeg|png|pdf)$");
    private final StudentDAO studentDAO = new StudentDAO();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        // 1. Validate request parameters
        String type = request.getParameter("type");
        int id;
        try {
            id = Integer.parseInt(request.getParameter("id"));
        } catch (NumberFormatException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid id");
            return;
        }
        if (!"photo".equals(type) && !"doc".equals(type)) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid type");
            return;
        }

        // 2. Get the stored file name from the database
        String fileName;
        try {
            Student s = studentDAO.findById(id);
            if (s == null) {
                response.sendError(HttpServletResponse.SC_NOT_FOUND);
                return;
            }
            fileName = "photo".equals(type) ? s.getPhotoFile() : s.getDocumentFile();
        } catch (SQLException e) {
            LOG.log(Level.SEVERE, "File lookup failed", e);
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            return;
        }
        if (fileName == null || !SAFE_NAME.matcher(fileName).matches()) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }
```

question3/src/main/java/com/kpr/portal/servlet/FileServlet.java (continued)

```
        // 3. Build the path and make sure it cannot escape the upload folder
        Path base = Paths.get(AppConfig.get("upload.dir"),
                "photo".equals(type) ? "photos" : "documents").toAbsolutePath().normalize();
        Path file = base.resolve(fileName).normalize();
        if (!file.startsWith(base) || !Files.isRegularFile(file) || Files.isSymbolicLink(file)) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        // 4. Send headers and stream the bytes
        String lower = fileName.toLowerCase(Locale.ROOT);
        String contentType = lower.endsWith(".png") ? "image/png"
                : lower.endsWith(".pdf") ? "application/pdf" : "image/jpeg";
        boolean download = "1".equals(request.getParameter("download"));

        response.setContentType(contentType);
        response.setContentLengthLong(Files.size(file));
        response.setHeader("X-Content-Type-Options", "nosniff");
        response.setHeader("Content-Disposition",
                (download ? "attachment" : "inline") + "; filename=\"" + fileName + "\"");

        try (InputStream in = Files.newInputStream(file);
             OutputStream out = response.getOutputStream()) {
            byte[] buffer = new byte[8192];
            int read;
            while ((read = in.read(buffer)) != -1) {
                out.write(buffer, 0, read);
            }
        }
    }
}
```

JSP image tags pointing to the servlet

Extract from students.jsp

```
<img src="${ctx}/file?type=photo&amp;id=${s.id}" alt="Photo of ...">
<a href="${ctx}/file?type=doc&amp;id=${s.id}" target="_blank">View document</a>
<a href="${ctx}/file?type=doc&amp;id=${s.id}&amp;download=1">Download</a>
```

15. Logout Servlet

question3/src/main/java/com/kpr/portal/servlet/LogoutServlet.java

```
package com.kpr.portal.servlet;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * Logs the user out. Only POST ends the session (a plain link or image
 * on another site cannot log the user out); GET just redirects.
 */
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();   // removes the user and all session data
        }
        response.sendRedirect(request.getContextPath() + "/login?loggedout=1");
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.sendRedirect(request.getContextPath() + "/students");
    }
}
```

16. Stylesheet

question3/src/main/webapp/css/style.css

```
/* css/style.css - Student Profile Portal */
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: #f1f5f9;
  color: #1f2937;
}

h1 { color: #1e3a8a; margin: 0 0 14px; }
.muted { color: #6b7280; }

/* ----- Top bar ----- */
.topbar {
  background: #1e3a8a; color: #fff;
  display: flex; justify-content: space-between; align-items: center;
  padding: 14px 24px; flex-wrap: wrap; gap: 10px;
}
.brand { font-size: 1.2rem; font-weight: 700; }
.user { display: flex; align-items: center; gap: 12px; }
form.inline { display: inline; margin: 0; }

main { max-width: 1100px; margin: 0 auto; padding: 24px 18px 40px; }
```

question3/src/main/webapp/css/style.css (continued)

```
/* ----- Buttons ----- */
.btn {
  display: inline-block; padding: 9px 18px; border: none; border-radius: 6px;
  background: #1e3a8a; color: #fff; font-size: .95rem; font-weight: 600;
  cursor: pointer; text-decoration: none;
}
.btn:hover { background: #1d4ed8; }
.btn-small { padding: 6px 14px; background: #ef4444; }
```

question3/src/main/webapp/css/style.css (continued)

```
.btn-small:hover { background: #dc2626; }
.btn-light { background: #e5e7eb; color: #374151; }
.btn-light:hover { background: #d1d5db; }

/* ----- Search ----- */
.search { display: flex; gap: 10px; margin-bottom: 14px; flex-wrap: wrap; }
.search input { flex: 1; min-width: 220px; padding: 10px 12px; border: 1.5px solid #cbd5e1; border-radius:
    6px; font-size: .95rem; }

/* ----- Profile cards ----- */
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); gap: 20px; margin-top:
    12px; }
.card {
  background: #fff; border-radius: 10px; padding: 20px; text-align: center;
  box-shadow: 0 2px 10px rgba(15,23,42,.1); border-top: 4px solid #2563eb;
}
.card .photo { display: flex; justify-content: center; margin-bottom: 12px; }
.card .photo img, .card .initial {
  width: 110px; height: 110px; border-radius: 50%; object-fit: cover; border: 3px solid #dbeafe;
}
.card .initial {
  background: #dbeafe; color: #1e3a8a; font-size: 2.6rem; font-weight: 700;
  display: flex; align-items: center; justify-content: center;
}
.card h2 { font-size: 1.1rem; margin: 0 0 10px; }
dl { text-align: left; margin: 0 0 12px; font-size: .9rem; }
dt { font-weight: 600; color: #475569; float: left; clear: left; width: 70px; }
dd { margin: 0 0 4px 74px; word-break: break-word; }
.links { display: flex; gap: 14px; justify-content: center; font-size: .9rem; }
.links a { color: #1d4ed8; font-weight: 600; }

/* ----- Alerts ----- */
.alert { padding: 11px 14px; border-radius: 6px; margin-bottom: 14px; font-weight: 600; }
.alert-error { background: #fee2e2; border: 1px solid #dc2626; color: #991b1b; }
.alert-success { background: #dcfce7; border: 1px solid #16a34a; color: #166534; }

/* ----- Login page ----- */
.login-page { min-height: 100vh; display: flex; align-items: center; justify-content: center; padding:
    20px; background: linear-gradient(135deg, #e8eef9, #f6f8fc); }
.login-box { width: 100%; max-width: 400px; background: #fff; border-radius: 10px; padding: 30px;
    box-shadow: 0 6px 24px rgba(15,23,42,.12); border-top: 5px solid #1e3a8a; }
.login-box h1 { text-align: center; font-size: 1.5rem; }
.login-box .muted { text-align: center; margin-top: 0; }
.login-box label { display: block; font-weight: 600; margin: 14px 0 5px; font-size: .92rem; }
.login-box input[type=text], .login-box input[type=password] {
  width: 100%; padding: 10px 12px; border: 1.5px solid #cbd5e1; border-radius: 6px; font-size: .95rem;
}
.login-box .btn { width: 100%; margin-top: 20px; text-align: center; padding: 11px; }
```

17. Explanation of the Code

• **DBConnection / AppConfig** – settings come from `db.properties` on the classpath; every DAO method opens a connection in try-with-resources so connections are always closed, even when an exception occurs.

• **PasswordUtil** – passwords are never stored in plain text. PBKDF2-HMAC-SHA256 with a random 16-byte salt and 120,000 iterations produces the stored value; `MessageDigest.isEqual()` compares hashes in constant time.

• **DAO classes** – use `PreparedStatement` with `?` placeholders, which prevents SQL injection, and map each `ResultSet` row to a `Student` or `User` object.

• **LoginServlet** – checks a CSRF token, validates input, verifies the hash, and returns the same error message whether the username or the password was wrong. After a successful login the old session is invalidated and a new one is created, which prevents session fixation.

• **AuthFilter** – intercepts `/students`, `/file` and `/logout`. `getSession(false)` avoids creating sessions for anonymous visitors; users without a `user` attribute are redirected to `/login`; protected responses are sent with no-cache headers.

• **StudentListServlet** – trims and limits the search text, calls the DAO, catches `SQLException`, and forwards the data to the JSP (JSP files are inside WEB-INF so they cannot be opened directly).

• **students.jsp** – `<c:forEach>` creates one card per student; `<c:choose>` shows the photo or an initial letter; `<c:if>` shows document links only when a document exists; `<c:out>` escapes data to prevent XSS.

• **FileServlet** – the browser sends only a student id and a type. The file name is read from the database, must match a whitelist regular expression, is resolved against the upload folder, normalised, and rejected unless `file.startsWith(base)`, it is a regular file and not a symbolic link. Content type is chosen from the extension, `nosniff` is set, and the bytes are streamed in 8 KB blocks.

• **LogoutServlet** – only a POST request invalidates the session, so another website cannot log the user out with a simple link or image.

• **web.xml** – session timeout (30 min), HttpOnly cookie, welcome file and custom error pages that hide technical details.

18. Result

The Java Servlet Student Profile Portal was developed using Servlets, JSP, JDBC, MySQL and Maven for Apache Tomcat 9. The application is written to authenticate users with hashed passwords and sessions, to protect its pages with a filter, to list student profiles dynamically with JSTL loops and conditions, to stream photos and documents through a servlet that validates file paths, and to log the user out safely.

19. Output Screenshots

Build and deploy the project as described in `question3/README.md`, then paste your own screenshots in the boxes below.

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 3.1 – Login page (and the invalid-login error message)

Figure 3.1 – Login page (and the invalid-login error message)

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 3.2 – Student profiles page after login

Figure 3.2 – Student profiles page after login

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 3.3 – Uploaded student images displayed through FileServlet

Figure 3.3 – Uploaded student images displayed through FileServlet

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 3.4 – Uploaded document opened / downloaded through FileServlet

Figure 3.4 – Uploaded document opened / downloaded through FileServlet

PASTE YOUR OUTPUT SCREENSHOT HERE

Figure 3.5 – Login page after logout ("You have been logged out")

Figure 3.5 – Login page after logout ("You have been logged out")

Conclusion

This assignment covered three stages of web application development. In Question 1 a registration form was built with HTML, CSS and JavaScript, showing how client-side validation gives users immediate feedback. In Question 2 a PHP and MySQL Student Management System implemented complete CRUD operations with prepared statements, input validation and secure, unique-name file uploads served through a protected script. In Question 3 a Java Servlet, JSP and JDBC portal added authentication, session handling, a filter, hashed passwords and path-validated streaming of images and documents on Apache Tomcat 9.

Across the three questions the same principles were applied: validate input on both client and server, never trust uploaded files, use prepared statements, escape all output, and keep sensitive files and pages behind controlled access. Together they give a practical understanding of how front-end, server-side scripting and Java enterprise technologies work with a relational database.